# The XFaceMaker Extensions

## Version 4.0

**Nova Software Labs**

Published by:


**Nova Software Labs**
8, rue de Hanovre
75002 Paris, France
Tel: +33 1 58 18 61 61
Fax: +33 1 58 18 61 60
email: sales@NovaSoftwareLabs.com or sales@nsl.fr.
Web: http://www.NovaSoftwareLabs.com or http://www.nsl.fr.



Document Number: XFMUIMSRefMan v. 7.1.1.
Document date: November 29, 2002

# Acknowledgments

The following people have contributed to the development of XFaceMaker:

Alain Bastard, Genevieve Beaujard, Nicole Brion, Alan Char, Eric Durocher, Marc Durocher, France Geze, Eric Herbaut, Elizabeth Huffer, Ion Filotti, Jérôme Maloberti, Sara Sautin, Eric Touchard, Philippe Volle.

Many thanks to all of them and to several others.

# Table of Contents

# Preface

The **XFaceMaker extensions** contain widgets designed by NSL, converters and convenience functions that extend the Face language's capabilities, in particular to access various resources of widgets from KLGroup.

The widgets designed by NSL provide useful and familiar data representations appropriate to a wide range of applications. These widgets are compatible with OSF/ Motif style guidelines. They are described in Chapter 1, The Control Library.

The XFaceMaker extension libraries described in this document are used to integrate widgets into the tool, provide structures and convenience functions, and to define new FACE types.

General extension libraries:

| | |
|---|---|
| **XnslStand** | Contains routines used by the NSL widgets running in applications built with **XFaceMaker**, in all modes: compiled (linked with libFm_c),intertpreted (linked with libFm), interactive interface editor (linked with libFm_e). |
| **XnslCtrl** | The library that contains the NSL Control widgets and converters. |
| **XnslTreeFm** | Contains routines for using the XnslTree widget with the XFaceMaker interactive interface editor, and in interpreted applications. |
| **XnslHtmlFm** | Contains routines for using the VistaPoint widget with the XFaceMaker interactive interface editor, and in interpreted applications. |
| **XnslFm** | Contains routines for using the NSL widgets (XnslControl, XnslDraw, XnslTree, VistaPoint) with the **XFaceMaker** interactive interface editor (linked with libFm_e), and in interpreted applications (linked with libFm). |

The libraries listed below are required to build interpreted applications or a new XFaceMaker executable including the KLGroup widgets. They contain the information relative to the functions that are accessible in the FACE language.

| | |
|---|---|
| **Xrt2dFm** | Used with the **Xrt/Graph** widget. |
| **Xrt3dFm** | Used with the **Xrt/3d** widget. |
| **XrtFieldFm** | Used with the **Xrt/Field** widget. |
| **XrtGearFm** | Used with the **Xrt/Gear** widget. |
| **XrtTableFm** | Used with the **Xrt/Table** widget. |

Additional libraries that are specific to the XRT widget integration are listed below:

| | |
|---|---|
| **XnslXrtPS** | Convenience functions common to the **XRT/Graph** and **XRT/3d** widgets. Required for all applications generated with XFaceMaker - compiled, interpreted, interface editor - that use either the **XRT/Graph** or the **XRT/3d** widgets. |
| **XnslGraph** | Convenience functions for the **Xrt/Graph** widget. This library is required for all three types of application generated with XFaceMaker: compiled, interpreted, interactive editor. |
| **Xnsl3d** | Convenience functions for the **XRT/3d** widget.This library is required for all three types of application generated with XFaceMaker: compiled, interpreted, interactive editor. |

This volume contains information concerning the convenience functions associated to the widget extensions, as well as a description of the NSL Control widgets. A few application examples are also given here.

The XnslDraw library, the XnslTree library and the XnslHtml (VistaPoint) library are described in separate documents.

The **XFaceMaker** documentation explains how to extend XFaceMaker and include your own widgets, or other third party widgets in a new XFaceMaker executable. Because applications generated by XFaceMaker are linked with all the libraries that have been specified when the extended executable is built, you may want to have several XFaceMaker executables, depending on the type of interface you are building, and which widgets you plan to use in these interfaces.

The extensions that are available in the product distribution are listed below. All extensions imply that the XnslStand library is added to XFaceMaker. XFaceMaker always includes the Control widgets.

Table 1: DefaultXFaceMaker Extensions

| Name | Include Files for FACE | Link Flag for Compiled appli. | Requirements |
|---|---|---|---|
| XnslDraw | Xnsl/DAll.h | -lXnslDraw -lXnslTiff -lXnslDf | Draw Library and XDraw license for development library, or decrypted run-time library |
| XnslHtml | Xnsl/Html.h | -lXnslHtml | XnslHtml Library and VistaPoint license for development library, or decrypted run-time library |
| XnslTree | Xnsl/Tree.h | -lXnslTree | XnslTree Library and XTree license for development library, or decrypted run-time library |
| XnslStand | Xnsl/Stand.h | -lXnslStand | included with all extensions. |
| Xrt3d | Xnsl/ Xrt3dFm.h | -lXnsl3d | XRT/3d widget library from KLGroup and license Includes XrtCommon |
| XrtGraph | Xnsl/Xrt-GraphFm.h | -lXnslGraph | XRT/graph widget library from KLGroup and license Includes XrtCommon |
| XrtCommon | Xnsl/XrtX-nslPs.h | -lXnslXrtPS | included with Xrt3d and Xrt-Graph - convenience functions shared between these widget extensions |
| XrtField | | | XRT/Field widget library from KLGroup and license |
| XrtGear | | | XRT/Gear widget library from KLGroup and license |
| XrtTable | | | XRT/Table widget library from KLGroup and license[1] |

1. If the function XrtTblEnableXrtField is attached to the FACE language, include also the XrtField extension. In the standard extension file, the function is not attached. To attach it, remove the comment.

### *Keyboard Conventions*

| Format | Example | Meaning |
|---|---|---|
| Key1--Key2 | Ctrl--A | Hold down the `Control` key as you press, then release key `A`. The `shift` key is not used. |
| Key1 Key2 | Esc a | Press and release the `Escape` key, then press and release key `a`. |
| | Esc A | Press and release the `Escape` key, then press and release key `a` while pressing the `shift` key. |

**Table 2: Keyboard Conventions**

For the `Esc` modifier the two keys are pressed in succession, whereas for the `Ctrl` modifier, the key is pressed *while* the `Ctrl` key is held down.

### *Typesetting Conventions*

| Type style | Used for |
|---|---|
| `typewriter` | Anything you should type as it appears, code, and key combinations. |
| *italic* | Mouse buttons and emphasis. |
| ***bold slant*** | Menu items and editing commands. |

**Table 3: Typesetting Conventions**

### *Technical Support*

Technical Support is available:

- Monday to Friday (except for French national holidays).
- 9.00 am 12.30 pm – 2.00 pm to 6.00 pm (MET). (5 pm on Friday).
- From French and English speaking support staff.
- By phone (33) [0]1.44.08.70.85 (direct) or (33) [0]1.44.08.70.80 (operator).
- By electronic mail addressed to `support@nsl.fr`
- By FAX (33) [0]1.44.08.70.81 addressed to Technical Support including your return FAX number, and a telephone number where you can be reached.

***Related Documents***

The following reference books may be useful.

James Getty, Robert Scheifler, Robert Newman, *Xlib – C Language X Interface*, MIT X Consortium Standard, X Version 11, Release 5, Massachusetts Institute of Technology, Cambridge, Massachusetts

Nova Software Labs, *XFaceMaker User's Guide*, Version 3.5

Nova Software Labs, *XDrawMaker User's Guide*, Version 3.5

Nova Software Labs, *XDraw Library Reference Manual,* Version 3.5

Nova Software Labs, *XTree Reference Manual,* Version 1.2

Nova Software Labs, *VistaPoint Reference Manual,* Version 1.2

Open Software Foundation, *OSF/Motif Programmer's Guide*, Release 1.2

Open Software Foundation, *OSF/Motif Programmer's Reference*, Rel. 1.2

Open Software Foundation, *OSF/Motif Style Guide*, Release 1.2

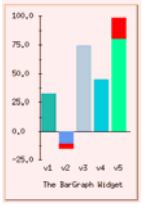
O'Reilly & Associates, *The Definitive Guides to the X Window System*

KL Group, Programmer's Guides and Reference Manuals for the XRT/PDS widget set.

# CHAPTER 1: The Control Library

The Control library consists of a set of control widgets. A brief illustrated description of these widgets is given in the first part of this chapter. It is followed by the detailed reference pages for each of the widgets.

**XnslBarGraph** an output widget designed to display a set of numeric values in a bar graph layout. It can be tailored to dynamically change the look of the bar graph in many different ways. An upper/lower threshold can be specified to invoke a warning callback when either threshold is crossed by one or several of the numeric values.




displayHorizontal False


displayHorizontal True

maxLimit = 80

minLimit = -10

warningColor = red

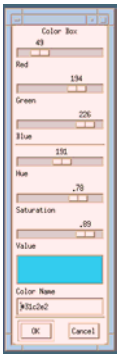
origin = 0

valueCount = 5

colorCount = 5

**XnslColorBox** an interactive widget that allows a user to define a new color, using the Red, Green, Blue or the Hue, Saturation, Intensity color representations. The color is diplayed in a rectangular pad, as it is being defined. The color value in the red,

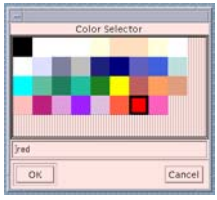
green, blue representation is printed in hexadecimal format, in the textField in the lower portion of the widget. A String containing this colorName is passed as the *call_data* of the OKCallback and the CancelCallback.





colorName = #31c2e2 #ff4008

**XnslColorSB** an interactive widget that allows a user to select a color in a color palette. It is best used as the child of an XmDialogShell, like the XmSelectionBox widget. The name of the color the user specified or selected is returned in the *call_data*. The set of colors displayed is controlled by the contents of the color file specified in the **XnslNcolorFile** resource. The widget implements a CancelCallback, an OKCallback, and a SelectCallback.


colorFile = /usr/lib/X11/rgb.txt mycolors.txt

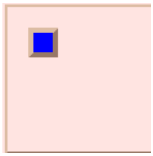
**XnslFontSelector** is an interactive widget that lets the user select a font. The fonts that are available onthe current display are listed in two scrolled lists, one for scalable fonts, the other for fixed fonts. The attributes of scalable fonts can be specified. A sample text in the lower part of the widget is displayed with the currently selected font. An OK and a Cancel button let the widget notify the application of the user's choice.


**XnslIndicator** an interactive widget which can be used as a multi-state button. Each time the user clicks in this widget, it can change to the next state, cycling through the available states. A different pixmap may be associated to each state.

numStates = 5
changeState = True

state = 0 1 2 3 4

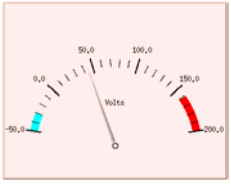
**XnslJoystick** an interactive widget that emulates the behavior of two orthogonal scrollbars whose slider sizes and positions can be changed. The joystick widget window contains a rectangular pad which can be moved (slider position) and resized (slider size) with the mouse. Resources and callbacks are provided to link the x- and y-position and the height and width of the pad to the application.



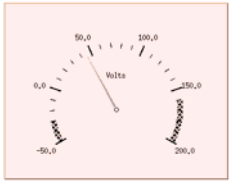
cursorStyle = cursor_solid cursor_pixmap cursor_out_line

**XnslMeter** an output widget that emulates an analog meter. It displays graphically a numeric value in a user interface. The meter looks like an arc of a circle on which a scale is drawn; a needle indicates the value.

value = 45.
title = Volts
divisions = 5
subDivisions = 5
maxLimit = 160.
minLimit = -30.
maximum = 200.
minimum = -50.




angle = 80
maxLimitColor = red
minLimitColor = cyan

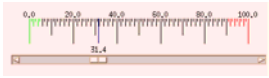



angle = 120
maxLimitPixmap =
minLimitPixmap =
wide_weave

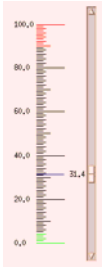
**XnslSlider** an interactive scale-like widget to control values from within a range. An upper/lower threshold can be specified to invoke a warning callback when either threshold is crossed. A set of divisions can be displayed. The color of the divisions extending beyond the thresholds can be specified separately.

showValue = true
showDivisions = true
showArrows = true
maxLimitColor = red
minLimitColor = green
divisionFormat = %.1f
decimalPoints = 1
maxLimit = 900
maximum = 1000
minLimit = 50

orientation = horizontal




vertical




**XnslStepper** an interactive widget to control a numerical value from within a range through a text input field and up/down arrows in auto-repeat mode.

maximum = 1000
value = 123

arrowPlacement = arrows_left


arrows_left_right


arrows_right

## 1.1 The Bar Graph Widget

### 1.1.1 Synopsis

*#include <Xnsl/BarGraph.h>*

### 1.1.2 Description

**XnslBarGraph** is an output widget designed to display a set of numeric values in a bar graph layout. It can be tailored to dynamically change the looks of the bar graph in many different ways.

An upper (**XnslNmaxLimit**) and lower (**XnslNminLimit**) threshold can be specified such that the **XnslNoutLimitCallback** is invoked when a numeric value in the graph moves beyond either threshold, and the **XnslNinLimitCallback** is invoked when a value that was outside the limits returns within bounds.

The graph is built by defining two orthogonal axes: the division axis and the bar axis.

The division axis corresponds to the scale of numeric values for the graph, ranging between the values **XnslNminimum** and **XnslNmaximum**. The axis is divided into a number of intervals equal to the value in **XnslNdivisions**. The numeric values at each interval boundary constitute the division legend, which may be displayed or not according to the **XnslNdisplayLegend** resource setting, in a format specified by the **XnslNdivisionFormat** resource. The numeric value specified by the **XnslNorigin** resource determines the position on the scale where the intersection with the bar axis takes place.

Lines corresponding to each numeric value in the displayed set are drawn across the bar axis. Since the division axis represents the origin, the bars are drawn on either side of it, depending on their numeric value relative to **XnslNorigin**. Each bar is separated from its neighbor by a number of pixels specified by the **XnslNinterval** resource.

The orientation of these two axes depends on the value of the **XnslNdisplayHorizontal** resource.

In vertical mode, the division axis is vertical and is placed at the left end of the window, possibly preceded by the division legends. The bar axis is placed horizontally to its right so that the bars are drawn vertically from the right of the division axis to the right end of the window. The graph may leave some space at the bottom of the window for the bar legends and, further down, for the bar title. The division unit, if defined, is placed at the top left corner of the window.

In horizontal mode, the division axis is horizontal and is placed at the bottom end of the window, possibly leaving some space below for the division legends and, further down, for the division unit. The bar axis is placed vertically above it so that the bars are drawn horizontally from just above the division axis up to the top end of the window. The graph may leave some space at the left side of the window for the bar legends and for the bar title at the topleft corner.

### 1.1.3 Classes

The **XnslBarGraph** widget class inherits behavior and resources from the **Core** and **Primitive** classes.

The class pointer is **xnslBarGraphWidgetClass**.

The class name is **XnslBarGraph**.

### 1.1.4 Resources

The **XnslBarGraph** widget class defines the new set of resources listed in the table below.

For resources that take floating-point values, the standard ***XtSetArg*** macro does not work properly. Instead, application code should use the macro ***XnslSetFloatArg*** to set floating point resource values.

C indicates that the resource can be set at creation time;

S indicates that the resource can be set using XtSetValues;

G indicates that the resource can be retrieved using XtGetValues.

**XnslBarGraph Resource Set**

| Name<br>Class | Default<br>Type | Access |
|---|---|---|
| XnslNbarShadowThickness<br>XnslCBarShadowThickness | 0<br>int | CSG |
| XnslNbarTitle<br>XnslCBarTitle | NULL<br>String | CSG |
| XnslNcolorCount<br>XnslCColorCount | 0<br>int | CSG |
| XnslNcolors<br>XnslCColors | NULL<br>Pixel * | CSG |

## XnslBarGraph Resource Set

| Name<br>Class | Default<br>Type | Access |
|---|---|---|
| XnslNdisplayAxis<br>XnslCDisplayAxis | True<br>Boolean | CSG |
| XnslNdisplayGrid<br>XnslCDisplayGrid | False<br>Boolean | CSG |
| XnslNdisplayHorizontal<br>XnslCDisplayHorizontal | False<br>Boolean | CSG |
| XnslNdisplayLegend<br>XnslCDisplayLegend | True<br>Boolean | CSG |
| XnslNdivisions<br>XnslCDivisions | 10<br>int | CSG |
| XnslNdivisionsFormat<br>XnslCDivisionsFormat | "%.2f"<br>String | CSG |
| XnslNemitWarning<br>XnslCEmitWarning | False<br>XtRBoolean | CSG |
| XnslNfont<br>XnslCFont | "fixed"<br>XFontStruct * | CSG |
| XnslNhorizontalMargin<br>XnslCHorizontalMargin | 2<br>int | CSG |
| XnslNinLimitCallback<br>XnslCInLimitCallback | NULL<br>XtRCallback | C |
| XnslNinterval<br>XnslCInterval | 4<br>int | CSG |
| XnslNlegends<br>XnslCLegends | NULL<br>String * | CSG |
| XnslNmaxLimit<br>XnslCMaxLimit | 10.0<br>float | CSG |
| XnslNmaximum<br>XnslCMaximum | 10.0<br>float | CSG |
| XnslNminLimit<br>XnslCMinLimit | -10.0<br>float | CSG |
| XnslNminimum<br>XnslCMinimum | -10.0<br>float | CSG |
| XnslNorigin<br>XnslCOrigin | 0.0<br>float | CSG |

## XnslBarGraph Resource Set

| Name<br>Class | Default<br>Type | Access |
|---|---|---|
| XnslNoutlimitCallback<br>XnslCOutLimitCallback | NULL<br>XtRCallback | C |
| XnslNpixmapCount<br>XnslCPixmapCount | 0<br>int | CSG |
| XnslNpixmaps<br>XnslCPixmaps | NULL<br>Pixmap * | CSG |
| XnslNsubDivisions<br>XnslCSubDivisions | 10<br>int | CSG |
| XnslNunitTitle<br>XnslCUnitTitle | NULL<br>String | CSG |
| XnslNvalueCount<br>XnslCValueCount | 0<br>int | CSG |
| XnslNvalues<br>XnslCValues | NULL<br>float * | CSG |
| XnslNverticalMargin<br>XnslCVerticalMargin | 2<br>int | CSG |
| XnslNwarningColor<br>XnslCWarningColor | “red”<br>Pixel | CSG |
| XnslNwarningPixmap<br>XnslCWarningPixmap | XmUNSPECIFIED_PIXMAP<br>Pixmap | CSG |

**XnslNbarShadowThickness**

Specifies the size of the border shadow drawn for each bar. Colors and pixmaps used to draw this shadow are taken from the XmPrimitive resources.

**XnslNbarTitle**

Specifies the string used as a legend for the set of bars, i.e., the string displayed as a title for the bar legends when **XnslNdisplayLegend** is *True*.

**XnslNcolorCount**

Specifies the number of colors in the bar color list. The value must be the number of items in XnslNcolors. If **XnslNcolorCount** is less than **XnslNvalueCount**, the colors are re-used as many times as needed. If the **XnslNpixmaps** resource has been set, the pixmap specification takes precedence over the color specification.

**XnslNcolors**

Points to an array of pixel values specifying the colors of the bars.

**XnslNdisplayAxis**

Specifies whether the graph axis. i.e. the base line at the value specified for the **XnslNorigin** resource, should be drawn or not.

**XnslNdisplayGrid**

If this resource is set to *True*, the bar graph is displayed in grid mode. In grid mode, for each (sub) division a (dashed) line is drawn from the division axis across the entire graph. Then another line is drawn between each bar, thus making a grid.

In non-grid mode, only a (very) small dash is drawn for each (sub) division on the division axis, and only if **XnslNdisplayLegend** is *True*. No lines are drawn between the bars.

**XnslNdisplayHorizontal**

Specifies whether the bars extend horizontally or vertically. If *True*, the bars are drawn horizontally rather than vertically, and the entire graph layout is changed accordingly.

**XnslNdisplayLegend**

Specifies whether or not legends are displayed with the graph; if *True*, the following legends are displayed:

- the XnslNbarTitle legend
- the XnslNunitTitle legend
- the division legends, i.e., the numeric values for each division on the division axis
- the small dashes across the division axis
- the XnslNlegends legends

**XnslNdivisions**

Specifies the number of divisions used across the division axis. The value range

defined by **XnslNminimum** and **XnslNmaximum** is divided into **XnslNdivisions** equal intervals, and the numeric value of each interval boundary is used as a division legend. The value must be greater than zero.

**XnslNdivisionFormat**

Specifies the string format used for the division legends. It has the same syntax as a printf(3) format string used to display a floating point value (e.g., "%.1f").

**XnslNemitWarning**

This is an interface debugging resource. If it is set toTrue, a warning is issued (through XtWarning) to signal any major errors in resource settings, such as: widget size specified is too small; incorrect combination of minimum, maximum, origin; negative count for colors, values, pixmaps; negative value for divisions or subdivisions; ... The default value for this resource is *False*.

**XnslNfont**

Specifies the font used for text display.

**XnslNinLimitCallback**

Specifies the list of callbacks called when any of the values in **`XnslNvalues`** that was outside the bounds crosses either the **`XnslNmaxLimit`** or the **`XnslNminLimit`** values. If several values cross either boundary, the callback is invoked once for each value. The widget may invoke this callback when **`XnslNvalues`**, **`XnslNmaxLimit`**, or **`XnslNminLimit`** is set. The reason field of the callback structure is **`XnslCR_VALUE_IS_IN_BOUNDS`**

**XnslNhorizontalMargin**

Specifies the width in pixels of the left and right margins of the XnslBarGraph widget, inside the Motif shadow. Nothing is drawn in these margins.

**XnslNinterval**

Specifies the size in pixels of the space inserted between two adjacent bars.

**XnslNlegends**

If **XnslNdisplayLegend** is *True*, points to an array of strings specifying the legends which will be displayed with each bar to describe each numeric value. The number of strings must be equal to **XnslNvalueCount**.

**XnslNmaxLimit**

Specifies the numeric value above which a warning must be issued. When any value is greater than **XnslNmaxLimit**, the corresponding bar is drawn in such a way that the part above **XnslNmaxLimit** is displayed using the **XnslNwarning-**

**Pixmap** pixmap or the **XnslNwarningColor** color if the pixmap is undefined. Also, the widget uses this value to determine when to invoke the **XnslNinLimitCallback** and the **XnslNoutLimitCallback**.

**XnslNmaximum**

Specifies the upper limit of the numeric values range, which is the maximum value that can be displayed.

**XnslNminLimit**

Specifies the numeric value below which a warning must be issued. When any value is less than or equal to **XnslNminLimit**, the corresponding bar is drawn in such a way that the part below **XnslNminLimit** is displayed using the **XnslNwarningPixmap** pixmap or the **XnslNwarningColor** color if the pixmap is undefined. Also, the widget uses this value to determine when to invoke the **XnslNinLimitCallback** and the **XnslNoutLimitCallback**.

**XnslNminimum**

Specifies the lower limit of the numeric values range, which is the minimum value that can be displayed.

**XnslNorigin**

Specifies the origin of the scale on the division axis. This is the numeric value between **XnslNminimum** and **XnslNmaximum** where the bar axis will be placed, and which will serve as the starting point for all the bar drawings.

**XnslNoutLimitCallback**

Specifies the list of callbacks called when any of the values in **`XnslNvalues`** that was within the bounds crosses either the **`XnslNmaxLimit`** or the **`XnslNminLimit`** values. If several values cross either boundary, the callback is invoked once for each value. The widget may invoke this callback when **`XnslNvalues`**, **`XnslNmaxLimit`**, or **`XnslNminLimit`** is set. The reason field of the callback structure is

**`XnslCR_VALUE_IS_MIN`** when a value becomes out of bounds because it lies beyond the **`XnslNminLimit`** value.

**`XnslCR_VALUE_IS_MAX`** when a value becomes out of bounds because it lies beyond the **`XnslNmaxLimit`** value.

**XnslNpixmapCount**

Specifies the number of pixmaps in the bar pixmap list. The value must be the number of items in **XnslNpixmaps**. If **XnslNpixmapCount** is less than **XnslNvalueCount** the pixmaps are reused as many times as needed.

**XnslNpixmaps**

Points to an array of pixmaps specifying the pixmaps used to draw the bars.

**XnslNsubDivisions**

Specifies the number of subdivision intervals inside one division interval. The value must be greater than zero. Specify *one* for no subdivisions.

**XnslNunitTitle**

Specifies the string used as a legend for the division axis to display the unit for the numeric values when **XnslNdisplayLegend** is *True*.

**XnslNvalueCount**

Specifies the number of values in the values list. The value must be the number of values in **XnslNvalues**.

**XnslNvalues**

Points to an array of floating point values specifying the numeric values displayed in the bar graph.

**XnslNverticalMargin**

Specifies the height in pixels of the top and bottom margins of the XnslBarGraph window, inside the Motif shadow. Nothing is drawn in these margins.

**XnslNwarningColor**

Specifies the color used to draw the bar parts which are outside the limits specified by **XnslNmaxLimit** and **XnslNminLimit**. This resource is ignored if **XnslNwarningPixmap** has been defined.

**XnslNwarningPixmap**

Specifies the pixmap used to draw the bar parts which are outside the limits specified by **XnslNmaxLimit** and **XnslNminLimit**.

### 1.1.5 Callback Information

The following structure is passed as the call_data arguments to each callback function:

```
typedef struct {
  int           reason;
  XEvent        *event;    /* always at 0 */
  int           index;
  float         value;
  float         previous_value;
} XnslBarGraphCallbackStruct;
```

The fields are defined as follows:

*reason* Indicates why the callback was invoked. Possible values are:

- XnslCR_VALUE_IS_IN_BOUNDS
- XnslCR_VALUE_IS_MIN
- XnslCR_VALUE_IS_MAX

*event* Is always **NULL** since the XnslBarGraph callbacks are not triggered by events.

*index* Indicates the index in the XnslNvalues array of the value which is out of bounds.

*value* Indicates the numeric value which is out of bounds.

*previous_value* Is the previous setting of the value at that index.

### 1.1.6 Translations

The XnslBarGraph widget specifies no default translations.

### 1.1.7 Actions

The XnslBarGraph widget defines no new actions.

**SEE ALSO**

XnslRegisterConverters XnslSetFloatArg

## 1.2 The Color Box Widget

### 1.2.1 Synopsis

```
#include <Xnsl/ColorBox.h>
```

### 1.2.2 Description

The **XnslColorBox** class consists of two sets of three scales that can be used to define a new color. One set of scales lets you define a color specifying Red, Green, Blue values. The other set of scales lets you define a color using the Hue, Saturation, Value values. As you modify the scale setting in one of the representations, the scales in the other representation are moved to reflect the change. Below the scales, a rectangular pad takes the new color as you specify it. A text field below the pad lets you enter a color name (actually the hexadecimal value of the desired color). Finally, the class contains an OK and a Cancel button.

### 1.2.3 Classes

The XnslColorBox class inherits resources and behavior from the **Core, Composite, Constraint, XmManager, XmBulletinBoard and XmForm** classes.

The class pointer is **xnslColorBoxWidgetClass**.

The class name is **XnslColorBox**

### 1.2.4 Resources

The **XnslColorBox** widget defines the new set of resources listed in the table below. Access codes use the same abbreviations as the OSF/Motif documentation:

C indicates that the resource can be set at creation time;

S indicates that the resource can be set using XtSetValues;

G indicates that the resource can be retrieved using XtGetValues.

**XnslColorBox Resource Set**

| Name<br>Class | Default<br>Type | Access |
|---|---|---|
| XnslNblue<br>XnslCBlue | dynamic<br>int | CSG |
| XnslNcancelCallback<br>XnslCCancelCallback | NULL<br>XtCallbackList | C |

## XnslColorBox Resource Set

| Name<br>Class | Default<br>Type | Access |
|---|---|---|
| XnslNcolorName<br>XnslCColorName | NULL<br>XtRString | CSG |
| XnslNgreen<br>XnslCGreen | dynamic<br>int | CSG |
| XnslNhue<br>XnslCHue | dynamic<br>int | CSG |
| XnslNintensity<br>XnslCIntensity | dynamic<br>int | CSG |
| XnslNokCallback<br>XnslCOkCallback | NULL<br>XtCallbackList | C |
| XnslNpixel<br>XnslCPixel | dynamic<br>Pixell | G |
| XnslNred<br>XnslCRed | dynamic<br>int | CSG |
| XnslNsaturation<br>XnslCSaturation | dynamic<br>int | CSG |

**XnslNblue**

Specifies the value of the ‘‘Blue’’ scale, with 0 <= *blue* <= 255.

**XnslNcancelCallback**

Specifies the list of callbacks called when the user activates the **Cancel** button. The value of **XnslNcolorName** is passed in the *call_data*.

**XnslNcolorName**

Specifies the name of the color in the text field. This is the hexadecimal value of the color in the Reg, Green, Blue representation. The value of this resource changes dynamically as the user interacts with the scales. The widget makes its own copy of the String when the application Sets or Gets this resource so the application may free its copy of the String when it no longer needs it.

**XnslNgreen**

Specifies the value of the ‘‘Green’’ scale, with 0 <= *green*<= 255.

**XnslNhue**

Specifies the value of the ‘‘Hue’’ scale, with 0 <= *hue*<= 360.

**XnslNintensity**

The value of the ‘‘Value’’ scale, with 0 <= *value* <= 100. Intensity is an integer resource; it is displayed with 2 decimal points to remind the user to think of this quantity as a percentage.

**XnslNokCallback**

Specifies the list of callbacks called when the user activates the **OK** button. The value of **XnslNcolorName** is passed in the *call_data*.

**XnslNpixel**

Specifies the value of the color. This is a read only resource, it is the value of the pixel that is allocated by the widget when it is created and freed when it is destroyed. Applications should not attempt to set this resource.

**XnslNred**

The value of the ‘‘Red’’ scale, with 0 <= *red* <= 255.

**XnslNsaturation**

The value of the ‘‘Saturation’’ scale, with 0 <= *saturation* <= 100. Saturation is an integer resource; it is displayed with 2 decimal points to remind the user to think of this quantity as a percentage.

### 1.2.5 Callback Information

The callbacks pass the color value (XnslNcolorName) in the *call_data*.

### 1.2.6 Translations

The widget defines no new translations; its components have their standard behavior.

**SEE ALSO**

XnslColorSB

## 1.3 The Color Selection Box Widget

### 1.3.1 Synopsis

```
#include <Xnsl/ColorSB.h>
```

### 1.3.2 Description

The **XnslColorSB** widget is a dialog widget that allows a user to select a color in a color palette. It is best used as the child of an XmDialogShell.

It includes the following:

- A label at the top of the box, to serve as a title field.
- A scrolled DrawingArea in which the colors defined in a color description file are displayed. Each color defined in the file is displayed in a square with the color or with a pattern that simulates intensity levels and hues, to limit color allocation requirements. When a color is selected (*Button1Down*), it is surrounded with a black square, and the SelectCallback is invoked.
- A SelectionBox that includes
  - a textField in which the user may enter a color name. *Return* in the textField invokes the widget's OK callback.
  - an OK button and a Cancel button that invoke the XnslColorSB widget's OKCallback and CancelCallback.

### 1.3.3 Classes

The XnslColorSB class inherits resources and behavior from the **Core**, **Composite**, **Constraint**, **XmManager**, **XmBulletinBoard** and **XmForm** classes.

The class pointer is **xnslColorSelectionBoxWidgetClass**.

The class name is **XnslColorSB**.

### 1.3.4 Resources

The **XnslColorSB** widget defines the new set of resources listed in the table below. Access codes use the same abbreviations as the OSF/Motif documentation:

C indicates that the resource can be set at creation time;

S indicates that the resource can be set using XtSetValues;

G indicates that the resource can be retrieved using XtGetValues.

## XnslColorSB Resource Set

| Name<br>Class | Default<br>Type | Access |
|---|---|---|
| XnslNcancelCallback<br>XnslCCancelCallback | NULL<br>XtCallbackList | C |
| XnslNcellsize<br>XnslCcellsize | 30<br>XtRInt | CSG |
| XnslNcolorFile<br>XnslCColorFile | "/usr/lib/X11/rgb.txt"<br>XtRString | CSG |
| XnslNcolorTable<br>XnslCColorTable | NULL<br>XnslRColorTable | G |
| XnslNcolumns<br>XnslCColumns | 10<br>XtRInt | G |
| XnslNokCallback<br>XnslCOkCallback | NULL<br>XtCallbackList | C |
| XnslNselectCallback<br>XnslCSelectCallback | NULL<br>XtCallbackList | C |
| XnslNselectedColor<br>XnslCSelectedColor | -1<br>XtRInt | CSG |
| XnslNselectedColorName<br>XnslCSelectedColorName | NULL<br>XtRString | CSG |

**XnslNcancelCallback**

Specifies the list of callbacks called when the user activates the **Cancel** button. The value of **XnslNselectedColorName** is passed in the *call_data*.

**XnslNcellsize**

Specifies the size of the squares representing each color in the palette.

**XnslNcolorFile**

Specifies the filename of the color description file, the default is "/usr/lib/X11/rgb.txt"

**XnslNcolorTable**

Specifies an entry in the colortable cache. For each color description file, there is a ColorTable structure. The table is used internally by the widget, applications should not try to set or even use this resource.

**XnslNcolumns**

The number of columns of the area containing the colors. The value is entirely determined by the widget's width, it is not intended for editing.

**XnslNokCallback**

Specifies the list of callbacks called when the user validates his choice, i.e. activates the **OK** button or enters *Return* in the text area. The value of **XnslNselectedColorName** is passed in the *call_data*.

**XnslNselectCallback**

Specifies the list of callbacks called when the user clicks on a color with mouse button 1. The value of **XnslNselectedColorName** is passed in the call_data.

**XnslNselectedColor**

Specifies the index of the selected color in the color table. This is not the Pixel value of the color, which can be obtained through the **XnslNselectedColorName** resource.

**XnslNselectedColorName**

Specifies the name of the selected color. To obtain the corresponding Pixel value, use a String to Pixel converter, e.g. *FaceConvertString*. The widget makes a copy of the color name, so the application can free its own string when it no longer needs it.

### 1.3.5 Callback Information

All the callbacks pass the name of the selected color in the *call_data*.

### 1.3.6 Translations

<Btn1Down>:Selects the color under the pointer and invokes the selectCallback

**SEE ALSO**

FaceConvertString, ColorBox

## 1.4 The Font Selector Box

### 1.4.1 Synopsis

```
#include <Xnsl/FontSelector.h>
```

### 1.4.2 Description

The **XnslFontSelector** class views the fonts available on the current display and lets the user select one amongst them.

An XnslFontSelector has six main areas :

- a scrollable list of font names for scalable fonts
- an attributes area with option menus and scale widgets
- a scrollable list of font names for non scalable fonts
- a text input field for displaying and editing font names
- a text input field for displaying text with the selected font
- two PushButtons, labeled **OK** and **Cancel**

The user can select a new font by scrolling either through the scalable fonts list or through the non scalable fonts list or by entering the font name directly into the text edit area. Selecting a new font causes the example text to change its font.

Selecting a non scalable font renders the attributes area insensitive. When a scalable font is selected, the user can modify its attributes using the option menus or scale elements. Any attribute modification changes the text edit font name and the example text area font.

The user can select a new font as many times as desired. The application is not notified until the user takes one of the actions below:

- Activates the **OK** PushButton
- Presses **KActivate** (usually *Return*) while the selection text edit area has the keyboard focus
- Double clicks or presses **KActivate** on an item in the non scalable font list

### 1.4.3 Classes

The XnslFontSelector class inherits resources and behavior from the **Core, Composite, Constraint, XmManager, XmBulletinBoard** and **XmForm** classes.

The class pointer is **xnslFontSelectorWidgetClass**.

The class name is **XnslFontSelector**.

### 1.4.4 Resources

The **XnslFontSelector** widget defines the new set of resources listed in the table below. Access codes use the same abbreviations as the OSF/Motif documentation:

C indicates that the resource can be set at creation time;

S indicates that the resource can be set using XtSetValues;

G indicates that the resource can be retrieved using XtGetValues.

**XnslFontSelector Resource Set**

| Name<br>Class | Default<br>Type | Access |
|---|---|---|
| XnslNexampleString<br>XnslCexampleString | "The quick brown fox\njumps over the lazy dog"<br>XmString | CSG |
| XnslNfontFamilyLabel<br>XnslCfontFamilyLabel | "Family"<br>XtRString | CSG |
| XnslNfontName<br>XnslCfontName | "fixed"<br>XtRString | CSG |
| XnslNfontOtherLabel<br>XnslCfontOtherLabel | "Other Fonts"<br>XtRString | CSG |
| XnslNfontPixelLabel<br>XnslCfontPixelLabel | "Size"<br>XtRString | CSG |
| XnslNfontScaleLabel<br>XnslCfontScaleLabel | "Size in Pixels"<br>XtRString | CSG |
| XnslNfontSelectorCallback<br>XnslCfontSelectorCallback | NULL<br>XtCallbackList | C |
| XnslNfontSetwidthLabel<br>XnslCfontSetwidthLabel | "Variation"<br>XtRString | CSG |
| XnslNfontSlantLabel<br>XnslCfontSlantLabel | "Slant"<br>XtRString | CSG |
| XnslNfontTitle<br>XnslCfontTitle | "Fonts Selector"<br>XtRString | CSG |
| XnslNfontWeightLabel<br>XnslCfontWeightLabel | "Weight"<br>XtRString | CSG |
| XnslNsetNameAndFont<br>XnslCSetNameAndFont | True<br>XtRBoolean | |

**XnslNexampleString**

Specifies the example text to display with the selected font.

**XnslNfontFamilyLabel**

Specifies the label of the font family list.

**XnslNfontName**

Specifies the name of the currently selected font.

**XnslNfontOtherLabel**

Specifies the label of the non scalable font list.

**XnslNfontPixelLabel**

Specifies the label of the size attribute option menu.

**XnslNfontScaleLabel**

Specifies the label of the scale widget which controls the size attribute along with the size option menu.

**XnslNfontSelectorCallback**

Specifies the list of callbacks called when the user activates the **OK** or the **Cancel** button.

**XnslNfontSetwidthLabel**

Specifies the label of the setwidth attribute option menu.

**XnslNfontSlantLabel**

Specifies the label of the slant attribute option menu.

**XnslNfontTitle**

Specifies the title displayed in the widget.

**XnslNfontWeightLabel**

Specifies the label of the weight attribute option menu.

**XnslNsetNameAndFont**

This resource is for internal use in the widget; it should not be modified by the application.

### 1.4.5 Callback Information

A pointer to the following structure is passed to the xnslNFontSelectorCallback. The *font_name* field is relevant only if *reason* is XnslCR_OK.

```
typedef struct {
        int                     reason;
        XEvent                  *event;
        String                  font_name;
       } xnslFontSelectorCallbackStruct;
```

*reason* Indicates why the callback was invoked.

- XnslCR_FS_CANCEL
- XnslCR_FS_OK

*event* Pointer to the XEvent that triggered the callback.

*font_name* Pointer to the string containing the selected font name. This field is meaningful only if *reason* is XnslCR_FS_OK.

### 1.4.6 Translations

The widget defines no new translations; its components have their standard behavior.

**SEE ALSO**

## 1.5 The Indicator Widget

### 1.5.1 Synopsis

*#include <Xnsl/Indicator.h>*

### 1.5.2 Description

**XnslIndicator** is a simple widget which can be used as a multi-state button. Each time the user clicks in this widget, it changes to the next state and displays a different image for each state.

### 1.5.3 Classes

The **XnslIndicator** inherits behavior and resources from **Core** and **Primitive**.

The class pointer is **xnslIndicatorWidgetClass**

The class name is **XnslIndicator**

### 1.5.4 Resources

The **XnslIndicator** widget class defines the new set of resources listed in the table below. Access codes use the same abbreviations as the OSF/Motif documentation:

C indicates that the resource can be set at creation time;

S indicates that the resource can be set using XtSetValues;

G indicates that the resource can be retrieved using XtGetValues.

**XnslIndicator Resource Set**

| Name<br>Class | Default<br>Type | Access |
|---|---|---|
| XnslNactivateCallback<br>XmCCallback | NULL<br>XtCallbackList | C |
| XnslNarmCallback<br>XmCCallback | NULL<br>XtCallbackList | C |
| XnslNchangeState<br>XnslCChangeState | False<br>XmRBoolean | CSG |
| XnslNnumStates<br>XnslCNumStates | 0<br>int | CSG |
| XnslNpixmap<br>XnslCPixmap | XmUNSPECIFIED_PIXMAP<br>Pixmap | CSG |

## XnslIndicator Resource Set

| Name<br>Class | Default<br>Type | Access |
|---|---|---|
| XnslNpixmaps<br>XnslCPixmaps | NULL<br>Pixmap * | CSG |
| XnslNstate<br>XnslCState | 0<br>int | CSG |

**XnslNactivateCallback**

Specifies a callback list which is called when the mouse Select button is released.

**XnslNarmCallback**

Specifies a callback list which is called when the mouse Select button is pressed.

**XnslNchangeState**

When true, the state of the widget is incremented by one unit whenthe **XnslNactivateCallback** is invoked. When the state reaches **XnslNnumStates** - 1 it is cycled back to zero.

**XnslNpixmap**

Specifies the default pixmap to be used by the indicator. It is the pixmap used to represent states for which a specific pixmap has not been provided.

**XnslNpixmaps**

Specifies an array of pixmaps to be used for the different states of the indicator. The number of pixmaps specified must be equal to or less than the value of the **XnslNnumStates** resource.

**XnslNstate**

Specifies the state of the indicator, a number that must lie within the range zero to **XnslNnumStates** - 1.

**XnslNnumStates**

Specifies the number of states of the indicator.

### 1.5.5 Callback Information

The following structure is passed as the call_data arguments to each callback function:

```
typedef struct {
        int             reason;
        XEvent          *event;
        int             state;
} XnslIndicatorCallbackStruct;
```

The fields are defined as follows:

*reason* Indicates why the callback was invoked. Possible values are:

- XnslCR_ARM
- XnslCR_ACTIVATE

*event* Points to the event that triggered the callback.

*state* Indicates the new state of the indicator.

### 1.5.6 Translations

The following table lists the default translations for the XnslIndicator widget:

| | |
|---|---|
| <Btn1Down> : | arm() |
| <Btn1Up> : | activate() |

### 1.5.7 Actions

The following actions are known to the XnslIndicator widget:

**arm**: Call the callbacks listed in the XnslNarmCallback resource.

**activate**: Change state if enabled. Call the callbacks listed in the XnslNactivateCallback resource.

**SEE ALSO**

XnslRegisterConverters

## 1.6 The Joystick Widget

### 1.6.1 Synopsis

```
#include <Xnsl/Joystick.h>
```

### 1.6.2 Description

**XnslJoystick** is a two-dimensional interactive widget which can be used to control two values through the same interface object. The joystick widget window contains a small pad which can be moved/resized with the mouse. Resources and callbacks are provided to link the x- and y-position of the square to application variables. This widget emulates the behavior of two orthogonal scrollbars whose slider sizes and positions can be changed.

### 1.6.3 Classes

The **XnslJoystick** inherits behavior and resources from **Core** and **Primitive**.

The class pointer is **xnslJoystickWidgetClass**.

The class name is **XnslJoystick**.

### 1.6.4 Resources

The **XnslJoystick** widget class defines the new set of resources listed in the table below.

C indicates that the resource can be set at creation time;

S indicates that the resource can be set using XtSetValues;

G indicates that the resource can be retrieved using XtGetValues.

**XnslJoystick Resource Set**

| Name<br>Class | Default<br>Type | Access |
|---|---|---|
| XnslNcursorBorderColor<br>XnslCCursorBorderColor | "green"<br>Pixel | CSG |
| XnslNcursorBorderPixmap<br>XnslCCursorBorderPixmap | XmUNSPECIFIED_PIXMAP<br>Pixmap | CSG |
| XnslNcursorBorderStyle<br>XnslCCursorBorderStyle | XnslBORDER_SHADOW_OUT<br>int | CSG |

## XnslJoystick Resource Set

| Name<br>Class | Default<br>Type | Access |
|---|---|---|
| XnslNcursorBorderWidth<br>XnslCCursorBorderWidth | 4<br>int | CSG |
| XnslNcursorColor<br>XnslCCursorColor | “blue”<br>Pixel | CSG |
| XnslNcursorHorizontalSize<br>XnslCCursorHorizontalSize | dynamic<br>int | CSG |
| XnslNcursorPixmap<br>XnslCCursorPixmap | XmUNSPECIFIED_PIXMAP<br>Pixmap | CSG |
| XnslNcursorStyle<br>XnslCCursorStyle | XnslCURSOR_SOLID<br>int | CSG |
| XnslNcursorVerticalSize<br>XnslCCursorVerticalSize | dynamic<br>int | CSG |
| XnslNdragCallback<br>XmCCallback | NULL<br>XtCallbackList | C |
| XnslNreleaseCallback<br>XmCCallback | NULL<br>XtCallbackList | C |
| XnslNresizeMode<br>XnslCResizeMode | XnslRESIZE<br>int | CSG |
| XnslNselectCallback<br>XmCCallback | NULL<br>XtCallbackList | C |
| XnslNvalueChangedCallback<br>XmCCallback | NULL<br>XtCallbackList | C |
| XnslNxMaxValue<br>XnslCXMaxValue | 100<br>int | CSG |
| XnslNxMinValue<br>XnslCXMinValue | 0<br>int | CSG |
| XnslNxValue<br>XnslCXValue | 50<br>int | CSG |
| XnslNyMaxValue<br>XnslCYMaxValue | 100<br>int | CSG |
| XnslNyMinValue<br>XnslCYMinValue | 0<br>int | CSG |
| XnslNyValue<br>XnslCYValue | 50<br>int | CSG |

**XnslNcursorBorderColor**

Specifies the color of the pad border if the **XnslNborderStyle** resource is set to **XnslBORDER_LINE** and the **XnslNcursorBorderPixmap** is set to **XmUNSPECIFIED_PIXMAP**.

**XnslNcursorBorderPixmap**

Specifies the pixmap with which the pad border is drawn if the **XnslNborder-Style** resource is set to **XnslBORDER_LINE**.

**XnslNcursorBorderStyle**

Specifies the style of the padborder. Possible values are:

- **XnslBORDER_SHADOW_IN**
- **XnslBORDER_SHADOW_OUT**
- **XnslBORDER_LINE**

**XnslNcursorBorderWidth**

Specifies the width of the pad border.

**XnslNcursorColor**

Specifies the color of the pad if **XnslNcursorStyle** is **XnslCURSOR_SOLID**.

**XnslNcursorHorizontalSize**

Specifies the width of the joystick pad, excluding the border.

**XnslNcursorPixmap**

Specifies the joystick pad pixmap if **XnslNcursorStyle** is **XnslCURSOR_PIXMAP**.

**XnslNcursorStyle**

Specifies the style of the joystick pad. Possible values are:

- **XnslCURSOR_OUT_LINE** nothing is drawn inside the pad.
- **XnslCURSOR_SOLID** the pad is filled with XnslNcursorColor.
- **XnslCURSOR_PIXMAP** the pad is filled with XnslNcursorPixmap.

**XnslNcursorVerticalSize**

Specifies the height of the joystick pad, excluding the border.

**XnslNdragCallback**

Specifies a callback list to be called when the user moves the mouse while button 1 is pressed after the **XnslNselectCallback** has been invoked and before the **XnslNreleaseCallback** is triggered. The callback reason is **XnslCR_DRAG**.

The pad changes position or size as the mouse is moved, depending on whether the drag occurs on the pad or on its border. Note that **XnslNresizeMode** must be set to **XnslRESIZE** to allow pad width or height changes during drag.

**XnslNreleaseCallback**

Specifies a callback list to be called when the user releases mouse button 1. The callback reason is **XnslCR_RELEASE**.

**XnslNresizeMode**

Specifies if the user is allowed to resize the pad with the mouse. Possible values are **XnslNO_RESIZE** or **XnslDO_RESIZE**.

**XnslNselectCallback**

Specifies a callback list to be called when the user presses mouse button 1 anywhere in the widget. The callback reason is **XnslCR_ARM**. The pad moves to the x, y position closest to the mouse cursor.

**XnslNvalueChangedCallback**

Specifies a callback list to be called when one of **XnslNxValue, XnslNyValue, XnslNcursorHorizontalSize**, or **XnslNcursorVerticalSize** changes due to user interaction. The callback reason is a combination of the following values:

- **XnslCR_X_VALUE_CHANGED**
- **XnslCR_Y_VALUE_CHANGED**
- **XnslCR_W_VALUE_CHANGED**
- **XnslCR_H_VALUE_CHANGED**

indicating which value(s) actually changed. The corresponding fields of the callback structure are set. Note that **XnslNresizeMode** must be set to **XnslRESIZE** to change the width or height of the pad.

**XnslNxMaxValue**

Specifies the x value corresponding to the maximum (right) position of the pad.

**XnslNxMinValue**

Specifies the x value corresponding to the minimum (left) position of the pad.

**XnslNxValue**

The x position of the left side of the joystick pad. This value must be in the range from **XnslNxMinValue** to **XnslNxMaxValue - XnslNcursorHorizontalSize**, or a warning is issued.

**XnslNyMaxValue**

Specifies the y value corresponding to the maximum (bottom) position of the pad.

**XnslNyMinValue**

Specifies the y value corresponding to the minimum (top) position of the pad.

**XnslNyValue**

The y position of the top side of the joystick pad. This value must be in the range from **XnslNyMinValue** to **XnslNyMaxValue - XnslNcursorVerticalSize**, or a warning is issued.

### 1.6.5 Callback Information

The following structure is passed as the call_data arguments to each callback function:

```
typedef struct {
        int             reason;
        XEvent          *event;
        int             x;
        int             y;
        int             width;
        int             height;
} XnslJoystickCallbackStruct;
```

The fields are defined as follows:

*reason* Indicates why the callback was invoked.

- XnslCR_ARM
- XnslCR_DRAG
- XnslCR_RELEASE
- a combination of
  - XnslCR_X_VALUE_CHANGED
  - XnslCR_Y_VALUE_CHANGED
  - XnslCR_W_VALUE_CHANGED
  - XnslCR_H_VALUE_CHANGEd

*event* Points to the button event that triggered the callback, or **NULL** if none.

*x* x position of the pad

*y* y position of the pad.

*width* width of the pad.

*height* height of the pad.

### 1.6.6 Constants

- XnslNcursorBorderStyle resource
  - XnslBORDER_SHADOW_IN
  - XnslBORDER_SHADOW_OUT
  - XnslBORDER_LINE
- XnslNcursorStyle resource
  - XnslCURSOR_OUT_LINE
  - XnslCURSOR_SOLID
  - XnslCURSOR_PIXMAP
- XnslNresizeMode resource
  - XnslNO_RESIZE
  - XnslDO_RESIZE

### 1.6.7 Translations

The **XnslJoystick** translations are listed below:

| | |
|---|---|
| <Btn1Down>: | Select() |
| <Btn1Motion>: | Drag() |
| <Btn1Up>: | Release() |

### 1.6.8 Actions

The following actions are known to the **XnslJoystick** widget:

**Select**: Move pad to mouse cursor position and call the XnslNselectCallback callback

**Drag:** Move/resize the pad, and call the XnslNdragCallback callback.

**Release:** Call the XnslNreleaseCallback callback.

**SEE ALSO**

XnslRegisterConverters

## 1.7 The Meter Widget

### 1.7.1 Synopsis

```
#include <Xnsl/Meter.h>
```

### 1.7.2 Description

**XnslMeter** is an output widget shaped like an analog meter. It displays graphically a numeric value in a user interface. The meter looks like an arc of a circle on which a scale is drawn; a needle indicates the value. To use the widget interactively rather than just as an output device, specify a translation table that associates the *move-needle* action to an event.

### 1.7.3 Classes

The **XnslMeter** inherits behavior and resources from **Core** and **Primitive**.

The class pointer is **xnslMeterWidgetClass**

The class name is **XnslMeter**

### 1.7.4 Resources

The **XnslMeter** widget class defines the new set of resources listed in the table below.

C indicates that the resource can be set at creation time;

S indicates that the resource can be set using XtSetValues;

G indicates that the resource can be retrieved using XtGetValues.

For resources that take floating-point values, the standard XtSetArg macro does not work properly. Instead, application code should use the macro **XnslSetFloatArg** to set floating point resource values.

**XnslMeter Resource Set**

| Name<br>Class | Default<br>Type | Access |
|---|---|---|
| XnslNangle<br>XnslCAngle | 0<br>int | CSG |
| XnslNautoScale<br>XnslCAutoScale | False<br>Boolean | CSG |

## XnslMeter Resource Set

| Name<br>Class | Default<br>Type | Access |
|---|---|---|
| XnslNdivisions<br>XnslCDivisions | 0<br>int | CSG |
| XnslNdivisionsFormat<br>XnslCDivisionsFormat | dynamic<br>String | CSG |
| XnslNdragCallback<br>XmCCallback | NULL<br>XtCallbackList | C |
| XnslNfloatUnits<br>XnslCFloatUnits | True<br>Boolean | CSG |
| XnslNfont<br>XnslCFont | “fixed”<br>XFontStruct * | CSG |
| XnslNleftNeedleColor<br>XnslCLeftNeedleColor | dynamic<br>Pixel | CSG |
| XnslNleftNeedlePixmap<br>XnslCLeftNeedlePixmap | dynamic<br>Pixmap | CSG |
| XnslNlineNeedle<br>XnslCLineNeedle | False<br>XtRBoolean | CSG |
| XnslNmaxCallback<br>XmCCallback | NULL<br>XtCallbackList | C |
| XnslNmaxLimit<br>XnslCMaxLimit | 0.0<br>float | CSG |
| XnslNmaxLimitColor<br>XnslCMaxLimitColor | dynamic<br>Pixel | CSG |
| XnslNmaxLimitPixmap<br>XnslCMaxLimitPixmap | dynamic<br>Pixmap | CSG |
| XnslNmaximum<br>XnslCMax | 1.0<br>float | CSG |
| XnslNminCallback<br>XmCCallback | NULL<br>XtCallbackList | C |
| XnslNminLimit<br>XnslCMinLimit | 0.0<br>float | CSG |
| XnslNminLimitColor<br>XnslCMinLimitColor | dynamic<br>Pixel | CSG |
| XnslNminLimitPixmap<br>XnslCMinLimitPixmap | dynamic<br>Pixmap | CSG |

## XnslMeter Resource Set

| Name<br>Class | Default<br>Type | Access |
|---|---|---|
| XnslNminimum<br>XnslCMin | 0.0<br>float | CSG |
| XnslNneedleWidth<br>XnslCNeedleWidth | 0<br>XtRDimension | CSG |
| XnslNrightNeedleColor<br>XnslCRightNeedleColor | dynamic<br>Pixel | CSG |
| XnslNrightNeedlePixmap<br>XnslCRightNeedlePixmap | dynamic<br>Pixmap | CSG |
| XnslNroundBounds<br>XnslCRoundBounds | True<br>Boolean | CSG |
| XnslNscaleFactor<br>XnslCScaleFactor | 0.0<br>float | CSG |
| XnslNsubDivisions<br>XnslCSubDivisions | 5<br>int | CSG |
| XnslNtitle<br>XnslCTitle | NULL<br>String | CSG |
| XnslNvalue<br>XnslCValue | 0.0<br>float | CSG |
| XnslNvalueChangedCallback<br>XmCCallback | NULL<br>XtCallbackList | C |
| XnslNxorMode<br>XnslCXorMode | True<br>Boolean | CSG |

### XnslNangle

Specifies the angle in degrees used to display one half of the arc of the scale. For example, a value of 90 will cause the meter to look like a half-circle. The angle must be non-negative and less than 180 degrees. If the angle is zero, the meter will compute the angle itself according to its dimensions.

### XnslNautoScale

Specifies whether the meter should perform automatic scaling, i.e., if it should automatically change its scale when the value is out of bounds. The default is *False*. Setting this resource to *True* affects the widget's behavior only if the **XnslNroundBounds** resource is *True*.

**XnslNdivisions**

Specifies the number of main divisions on the meter scale. If a value of 0 is specified, the number of divisions is computed automatically so that the interval between two divisions is a power of 10. Otherwise, the interval is the maximum minus the minimum divided by the number of divisions. The default is zero.

**XnslNdivisionsFormat**

Specifies the format used to print the division values, using the same format as printf(3). If **XnslNfloatUnits** is *True*, the default is "%1.1f", otherwise "%d". The format specified in this resource is used also to display the scaleFactor legend.

**XnslNdragCallback**

Specifies a callback list called by the *move-needle* action if the user event is a MotionNotify event. (This action is not bound to any event in the default translation table.) The callback reason is **XmCR_DRAG**.

**XnslNfloatUnits**

Specifies whether the scale values should be displayed as floating-point values (with a decimal point) or not. The default is *True*.

**XnslNfont**

Specifies the font to be used when displaying the scale values.

**XnslNleftNeedleColor**

Specifies the color used to draw the "left" side of the needle. This is initialized to the color of the top of the 3-D shadow of the widget.

**XnslNleftNeedlePixmap**

Specifies the Pixmap used to draw the "left" side of the needle. This is initialized to the Pixmap of the top of the 3-D shadow of the widget.

**XnslNlineNeedle**

If *True*, specifies that the needle should be drawn as a simple line whose width is **XnslNneedleWidth**, whose color is **XnslNleftNeedleColor,** whose pixmap is **XnslNleftNeedlePixmap.** If *False* (the default), the needle emulates the shape of an anolog meter needle with two back to back triangles.

**XnslNmaxCallback**

Specifies a callback list to be called when the meter value is greater than **XnslNmaxLimit**. The reason is **XnslCR_MAX_LIMIT**.

**XnslNmaxLimit**

Specifies a maximum warning value to display on the scale. The limit is displayed using the **XnslNmaxLimitColor**, or the **XnslNmaxLimitPixmap**.

**XnslNmaxLimitColor**

Specifies the color to be used to draw the maximum limit of the scale. This resource is ignored if **XnslNmaxLimitPixmap** has been set.

**XnslNmaxLimitPixmap**

Specifies the Pixmap used to draw the maximum limit of the scale.

**XnslNmaximum**

Specifies the maximum value of the meter. If **XnslNroundBounds** is *True*, this value is automatically rounded to the nearest superior multiple of a power of 10 (e.g., 22.5 is rounded to 30; 347.6 is rounded to 400). If **XnslNdivisions** is *zero*, the divisions of the scale correspond to the power of 10 of which this value is a multiple.

**XnslNminCallback**

Specifies a callback list to be called when the meter value is less than **XnslNminLimit**. The callback reason is **XnslCR_MIN_LIMIT**.

**XnslNminLimit**

Specifies a minimum warning value to display on the scale. The warning zone is displayed using the **XnslNminLimitColor**, or the **XnslNminLimitPixmap**.

**XnslNminLimitColor**

Specifies the color to be used to draw the minimum limit of the scale. This resource is ignored if **XnslNminLimitPixmap** has been set.

**XnslNminLimitPixmap**

Specifies the Pixmap used to draw the minimum limit of the scale.

**XnslNminimum**

Specifies the minimum value of the meter scale. If **XnslNroundBounds** is *True*, this value is automatically rounded to the nearest inferior multiple of a power of 10 (e.g., 22.5 is rounded to 20; 347.6 is rounded to 300). If **XnslNdivisions** is *zero*, the divisions of the scale correspond to the power of 10 of which this value

is a multiple.

**XnslNneedleWidth**

Specifies the width of the needle drawn if **XnslNlineNeedle** is *True*. This resource is ignored otherwise.

**XnslNrightNeedleColor**

Specifies the color used to draw the "right" side of the needle. This is initialized to the color of the bottom of the 3-D shadow of the widget.

**XnslNrightNeedlePixmap**

Specifies the Pixmap used to draw the "right" side of the needle. This is initialized to the Pixmap of the bottom of the 3-D shadow of the widget.

**XnslNroundBounds**

Specifies whether the meter should round the maximum and minimum values to the nearest power of 10. The default is *True*.

**XnslNscaleFactor**

Specifies a scale factor to apply to divisions. For example, if **XnslNscaleFactor** is *100*, a division of value 1000 will be displayed as 10, and the scale factor "x 100" will be displayed near the center of the meter. A factor of *zero* means do not display any factor. A factor of 1 will be displayed as "x 1". A factor of 0.25 will be displayed as ": 4"

**XnslNsubDivisions**

Specifies the number of secondary divisions on the meter scale. A value of *zero* or *one* means no secondary divisions. If **XnslNsubDivisions** is too big to be displayed, no subdivisions are displayed. The default is *5*.

**XnslNtitle**

Specifies a string to be displayed on the meter. The title is displayed to the left of the scale factor, if any.

**XnslNvalue**

Specifies the value pointed to by the needle. If the value is greater than **XnslNmaximum** or less than **XnslNminimum**, and **XnslNautoScale** is *True*, the scale is extended automatically (and the scale's corresponding limit is changed).

**XnslNvalueChangedCallback**

Specifies a callback list to be called every time the **XnslNvalue** resource changes. The reason is **XmCR_VALUE_CHANGED**.

**XnslNxorMode**

Specifies whether the needle should be drawn using XORmode. If this resource is *True*, changing **XnslNvalue** will cause only the needle to move. If this is *False*, the whole meter will be redrawn.

### 1.7.5 Callback Information

The following structure is passed as the call_data arguments to each callback function:

```
typedef struct {
        int             reason;
        XEvent          *event;
        float           value;
} XnslMeterCallbackStruct;
```

The fields are defined as follows:

*reason* Is the reason for the callback.

- XmCR_DRAG
- XnslCR_MAX_LIMIT
- XnslCR_MIN_LIMIT
- XmCR_VALUE_CHANGED

*event* Points to the button event that triggered the callback, or **NULL** if none.

*value* Is the **XnslNvalue** resource at the time the callback function is called.

### 1.7.6 Translations

The XnslMeter widget class does not add or change any translations.

### 1.7.7 Actions

The following action is defined by the XnslMeter widget class:

**move-needle**: Move the needle to the position defined by the event. It can be bound to any mouse event (i.e., ButtonPress,ButtonRelease, or MotionNotify).

**SEE ALSO**

XnslRegisterConverters, XnslSetFloatArg

## 1.8 The Slider Widget

### 1.8.1 Synopsis

```
#include <Xnsl/Slider.h>
```

### 1.8.2 Description

**XnslSlider** an interactive scale-like widget to control values from within a range. An upper/lower threshold can be specified to invoke a warning callback when either threshold is crossed. A set of divisions can be displayed. The color of the divisions extending beyond the thresholds can be specified separately.

### 1.8.3 Classes

**XnslSlider** inherits behavior and resources from **Core**, **Composite**, **Constraint**, **Manager**, and **Scale**.

The class pointer is **xnslSliderWidgetClass**.

The class name is **XnslSlider**.

### 1.8.4 Resources

The **XnslSlider** widget class defines the new set of resources listed in the table below. Access codes use the same abbreviations as the OSF/Motif documentation:

C indicates that the resource can be set at creation time;

S indicates that the resource can be set using XtSetValues;

G indicates that the resource can be retrieved using XtGetValues.

**XnslSlider Resource Set**

| Name<br>Class | Default<br>Type | Access |
|---|---|---|
| XnslNdivisions<br>XnslCDivisions | 5<br>int | CSG |
| XnslNdivisionsFormat<br>XnslDivisionsFormat | "%3d"<br>String | CSG |
| XnslNlimitPixmap<br>XnslRLimitPixmap | XmUNSPECIFIED_PIXMAP<br>Pixmap | CSG |
| XnslNmaxCallback<br>XnslCMaxCallback | NULL<br>XtCallbackList | C |

## XnslSlider Resource Set

| Name<br>Class | Default<br>Type | Access |
|---|---|---|
| XnslNmaxLimit<br>XnslMaxLimit | 100<br>int | CSG |
| XnslNminCallback<br>XnslCMinCallback | NULL<br>XtCallbackList | C |
| XnslNminLimit<br>XnslMinLimit | 0<br>int | CSG |
| XnslNshowArrows<br>XnslCShowArrows | True<br>Boolean | CSG |
| XnslNshowDivision<br>XnslCShowDivision | True<br>Boolean | CSG |
| XnslNsubDivisions<br>XnslCSubDivisions | 2<br>int | CSG |
| XnslNsubSubDivisions<br>XnslCSubSubDivisions | 5<br>int | CSG |
| XnslNsubSubSubDivisions<br>XnslCSubSubSubDivisions | 2<br>int | CSG |

**XnslNdivisions**

Specifies the number of intervals into which the range between **XmNmaximum** and **XmNminimum** is divided.

**XnslNdivisionsFormat**

Specifies a printf(3) style format used to display the value of each division, e.g. "%5.1f".

**XnslNlimitPixmap**

Specifies a pixmap used as a stipple to display the divisions on either end of the slider, i.e., before the **XnslNminLimit** and after the **XnslNmaxLimit**.

**XnslNmaxCallback**

Specifies the list of callbacks called when the slider crosses the value of **XnslNmaxLimit**. The reason is either **XnslCR_ENTER_MAX** or **XnslCR_LEAVE_MAX** as appropriate.

**XnslNmaxLimit**

Specifies a value that, when crossed, invokes the **XnslNmaxCallback**.

**XnslNminCallback**

Specifies the list of callbacks called when the slider crosses the value of **XnslNminLimit**. The reason is either **XnslCR_ENTER_MIN** or **XnslCR_LEAVE_MIN** as appropriate.

**XnslNminLimit**

Specifies a value that, when crossed, invokes the **XnslNminCallback**.

**XnslNshowArrows**

Specifies whether or not the arrows are displayed at either end of the slider trough.

**XnslNshowDivisions**

Specifies whether or not the divisions are displayed.

**XnslNsubDivisions**

Specifies the number of intervals into which each division is divided.

**XnslNsubSubDivisions**

Specifies the number of intervals into which each subdivision is divided.

**XnslNsubSubSubDivisions**

Specifies the number of intervals into which each sub-subdivision is divided.

### 1.8.5 Callback Information

The following structure is passed as the call_data argument of the callback function:

```
typedef struct {
        int             reason;
        XEvent          *event;
        int             value;
} XnslSliderCallbackStruct;
```

The fields are defined as follows:

*reason* Indicates why the callback was invoked

- XnslCR_ENTER_MAX
- XnslCR_LEAVE_MAX
- XnslCR_ENTER_MIN
- XnslCR_LEAVE_MIN

*event* Points to the event that triggered the callback, or **NULL** if none.

*value* Is the **XmNvalue** resource at the time the callback function is called.

### 1.8.6 Translations

The XnslSlider widget specifies no default translations.

### 1.8.7 Actions

The XnslSlider widget defines no new actions.

**SEE ALSO**

XnslRegisterConverters

## 1.9 The Stepper Widget

### 1.9.1 Synopsis

```
#include <Xnsl/Stepper.h>
```

### 1.9.2 Description

**XnslStepper** consists of an XnslText widget with two OSF/Motif XmArrowButtons, and can be used to input and/or output a numeric value. The arrow buttons can be used to increase and decrease the value by clicking in the top arrow and bottom arrow respectively. The arrow buttons work in “auto repeat” mode: the value keeps increasing or decreasing while the user keeps the mouse button pressed.

### 1.9.3 Classes

The **XnslStepper** inherits behavior and resources from the **Core**, **Composite**, **Constraint**, **XmManager**, **XmBulletinBoard** and **XmForm** classes.

The class pointer is **xnslStepperWidgetClass**

The class name is **XnslStepper**

### 1.9.4 Resources

The **XnslStepper** widget class defines the new set of resources listed in the table below. Access codes use the same abbreviations as the OSF/Motif documentation:

C indicates that the resource can be set at creation time;

S indicates that the resource can be set using XtSetValues;

G indicates that the resource can be retrieved using XtGetValues

**XnslStepper Resource Set**

| Name<br>Class | Default<br>Type | Access |
|---|---|---|
| XnslNactivateCallback<br>XtCCallback | NULL<br>XtCallbackList | C |
| XnslNarrowPlacement<br>XnslCArrowPlacement | XnslARROW_LEFT<br>int | CSG |
| XnslNautoRepeat<br>XnslCAutoRepeat | True<br>Boolean | CSG |

## XnslStepper Resource Set

| Name<br>Class | Default<br>Type | Access |
|---|---|---|
| XnslNfont<br>XnslCFontList | 0<br>XmFontList | CSG |
| XnslNincrement<br>XnslCIncrement | 1<br>int | CSG |
| XnslNmaximum<br>XnslCMaximum | 1<br>int | CSG |
| XnslNminimum<br>XnslCMinimum | 100<br>int | CSG |
| XnslNmodifyVerifyCallback<br>XtCCallback | NULL<br>XtCallbackList | C |
| XnslNrepeatDelay<br>XnslCRepeatDelay | 500<br>int | CSG |
| XnslNrepeatInterval<br>XnslCRepeatInterval | 100<br>int | CSG |
| XnslNtimeoutId<br>XnslCTimeoutId | 0<br>TimeOut * | G |
| XnslNvalue<br>XnslCValue | 1<br>int | CSG |
| XnslNvalueChangedCallback<br>XtCCallback | NULL<br>XtCallbackList | C |

**XnslNactivateCallback**

Specifies the list of callback functions to be called when the text field is activated, usually by typing a Return. The callback data is the same as for the **XmText** widget.

**XnslNarrowPlacement**

Specifies how the arrow button will be arranged:

- **XnslARROWS_LEFT** The arrow buttons are both on the left-hand side, one above the other.
- **XnslARROWS_LEFT_RIGHT** The arrows are on each side of the text field.
- **XnslARROWS_RIGHT** The arrow buttons are both on the right-hand side, one above the other.

**XnslNautoRepeat**

Specifies whether or not the arrow buttons should auto-repeat.

**XnslNfontList**

Specifies the font list to be used in the text field.

**XnslNincrement**

Specifies the value by which to decrease or increase the numeric value when using the arrow buttons.

**XnslNmaximum**

Specifies the maximum numeric value of the stepper. If the user tries to increase the value above this maximum value, the stepper value will not change.

**XnslNminimum**

Specifies the minimum numeric value of the stepper. If the user tries to decrease the value below this minimum value, the stepper value will not change.

**XnslNmodifyVerifyCallback**

Specifies the list of callback functions to be called to verify a change in the text input field. This is called from the callback list specified in the **XmNmodifyVerifyCallback** resource of the stepper's text field. The callback data is the same as for the **XmText** widget.

**XnslNrepeatDelay**

Specifies the delay in milliseconds before starting to auto repeat for the arrow buttons.

**XnslNrepeatInterval**

Specifies the interval in milliseconds between increments for the arrow buttons when auto-repeating.

**XnslNtimeoutId**

Contains the Xt time-out identifier used to implement the arrow buttons' auto-repeat. This can be used by the application to stop the auto-repeat. See the **XmArrowButton** for details.

**XnslNvalue**

Specifies the numeric value to display.

**XnslNvalueChangedCallback**

Specifies the list of callback functions to be called when the numeric value of the stepper changes. The callback data is the new numeric value, not a pointer to a data structure.

### 1.9.5 Callback Information

See the documentation of the **XmText** widget for a description of the **XmAnyCallbackStruct** used by the **XnslNactivateCallbacklist** and the **XmTextVerifyCallbackStruct** used by the **XnslNmodifyVerifyCallback**.

### 1.9.6 Translations

XnslStepper specifies no default translations.

### 1.9.7 Actions

XnslStepper defines no new actions.

## SEE ALSO

XnslRegisterConverters

# CHAPTER 2: The Converter Functions

The following converter functions are defined in the libXnslCtrl.a or libXnslCtrl.so library. Their prototypes and the **XnslSetFloatArg** macro are in the Types.h file.

## NAME

**XnslRegisterColorPixmapConverter**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslCtrl [libraries]
#include <Xnsl/Types.h>

XnslRegisterColorPixmapConverter (type, primitive)
String              type;
Boolean             primitive;
```

## DESCRIPTION

**XnslRegisterColorPixmapConverter** registers a converter from a string containing the name of a pixmap file to an actual pixmap. The argument *type* contains the name of the pixmap representation type. The argument *primitive* determines which additional arguments to supply to the converter. If *primitive* is true, the converter receives arguments appropriate for widgets subclassed from the type **XmPrimitive**, otherwise the converter receives only those arguments for widgets subclassed from the type **Core**. The converter uses the environment variable **XBMLANGPATH** when searching for pixmaps.

## EXAMPLE

## SCOPE

Application.

## SEE ALSO

## NAME

**XnslRegisterConverters**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslCtrl [libraries]
#include <Xnsl/Types.h>

XnslRegisterConverters ();
```

## DESCRIPTION

**XnslRegisterConverters** registers resource converters for special types used by widgets in the Extension Libraries. The application may also register converters for the types individually. All converters convert from a string to the representation type. The initialization routines for the **NSL Widgets** automatically register converters. The programmer needs to call **XnslRegisterConverters** only when linking C code generated by **XFaceMaker** with the **Fm_c** library.

## EXAMPLE

## SCOPE

Application.

## SEE ALSO

**NAME**

**XnslRegisterFloatTableConverter**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnslCtrl [libraries]
#include <Xnsl/Types.h>

XnslRegisterFloatTableConverter ();
```

**DESCRIPTION**

**XnslRegisterFloatTableConverter** registers a converter from a string containing floating point values separated by commas to an array of values of type float. The type name is **XnslFloatTable**.

**EXAMPLE**

**SCOPE**

Application.

**SEE ALSO**

## NAME

**XnslRegisterPixelTableConverter**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslCtrl [libraries]
#include <Xnsl/Types.h>

XnslRegisterPixelTableConverter ();
```

## DESCRIPTION

**XnslRegisterPixelTableConverter** registers a converter from a string containing color names separated by commas to an array of pixel values. A color name may be an RGB value preceded by a pound sign (#). The type name is **XnslPixelTable**.

## EXAMPLE

## SCOPE

Application.

## SEE ALSO

**NAME**

**XnslRegisterPixmapTableConverter**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnslCtrl [libraries]
#include <Xnsl/Types.h>

XnslRegisterPixmapTableConverter();
```

**DESCRIPTION**

**XnslRegisterPixmapTableConverter** registers a converter from a string containing pixmap file names separated by commas to an array of pixmap Ids. The type name is **XnslPixmapTable**.

**EXAMPLE**

**SCOPE**

Application.

**SEE ALSO**

## NAME

**XnslRegisterStringTableConverter**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslCtrl [libraries]
#include <Xnsl/Types.h>

XnslRegisterStringTableConverter ();
```

## DESCRIPTION

**XnslRegisterStringTableConverter** registers a converter from a string containing string values separated by commas to an array of values of type string. The type name is **XnslStringTable**.

## EXAMPLE

## SCOPE

Application.

## SEE ALSO

**NAME**

**XnslSetFloatArg**

**SYNOPSIS**

```
#include <Xnsl/Types.h>

XnslSetFloatArg (arg, resource_name, value)
Arg          arg;
String       resource_name;
float        value;
```

**DESCRIPTION**

The **XnslSetFloatArg** macro should be used instead of **XtSetArg** when setting float resources.

| | |
|---|---|
| *arg* | Specifies the Arg structure to set |
| *resource_name* | Specifies the name of the resource |
| *value* | Specifies the value of the resource |

**EXAMPLE**

Read and set **XnslNmaxLimit** in an XnslBarGraph widget.

```
void maxL(Widget wbarG, int up) {
   Arg args[5];
   int n;
   float maxlimit;
   n = 0;
   XtSetArg(args[n], XnslNmaxLimit, &maxlimit);
   n++;
   XtGetValues (wbarG, args, n);
   printf("current maxlimit = %.1f\n", maxlimit);
   if (up)
       { maxlimit += 20.0; }
   else
       { maxlimit -= 20.0; }
   n = 0;
   XnslSetFloatArg(args[n],XnslNmaxLimit,maxlimit);
    n++;
    XtSetValues (wbarG, args, n);
}
```

**SCOPE**

Application

# CHAPTER 3: FACE Language Support

The NSL Extension Libraries attach functions and define variables and structures for use when programming in the **FACE** language. You may refer to these functions and variables in **FACE** scripts both when building your interface and at run time, if your application interprets XFaceMaker files, and when compiling **FACE** scripts.

Several libraries are used to integrate widgets:

- The **XnslFm** library integrates some of the NSL widgets (XnslDraw, XnslControl). It provides the structures, convenience functions, and constants required for XFaceMaker to interpret the data for the XnslDraw widget and the set of NSL Control widgets.
- The **XnslHtmlFm** library integrates the NSL VistaPoint widget. This library is generated by **xfmextend** from the .face widget description file. It provides the structures, convenience functions, and constants required for XFaceMaker to interpret VistaPoint widget data.
- The **XnslTreeFm** library integrates the NSL XnslTree widget. This library is generated by **xfmextend** from the .face widget description file. It provides the structures, convenience functions, and constants required for XFaceMaker to interpret XnslTree widget data.
- The **Xrt2dFm** library integrates the XRT/graph widget. This library is generated by **xfmextend** from the .face widget description file. It provides the structures, XRT/graph convenience functions, representation types and constants required for XFaceMaker to interpret XRT/graph widget data.
- The **Xrt3dFm** library integrates the Xrt3d widget. This library is generated by **xfmextend** from the .face widget description file. It provides the structures, XRT/3d convenience functions, representation types and constants required for XFaceMaker to interpret XRT/3d widget data.
- The **XrtTableFm** library integrates the XRT/table and XRT/label widgets. This library is generated by **xfmextend** from the .face widget description file. It provides the structures, XRT/table convenience functions, representation types and constants required for XFaceMaker to interpret XRT/table widget data.
- The **XrtGearFm** library integrates the XrtGear widgets. This library is generated by **xfmextend** from the .face widget description file. It provides the structures, XRT/Gear convenience functions, representation types and constants required

for XFaceMaker to interpret XRT/Gear widget data.

- The **XrtFieldFm** library integrates the XrtField widgets. This library is generated by **xfmextend** from the .face widget description file. It provides the structures, XRT/Field convenience functions, representation types and constants required for XFaceMaker to interpret XRT/Field widget data.

The following pages list the available initialization functions.

## NAME

**XnslInit3dLibrary**

## SYNOPSIS

```
cc [flags] files [libraries] -lXrt3dFm [libraries]

void XnslInit3dLibrary ()
```

## DESCRIPTION

The function **XnslInit3dLibrary** initializes the Xrt3d library with the **XFaceMaker** graphic interface editor or interpreted **XFaceMaker** applications.

## EXAMPLE

## SCOPE

Application.

## SEE ALSO

## NAME

**XnslInitControlLibrary**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslFm [libraries]

int XnslInitControlLibrary ()
```

## DESCRIPTION

The function **XnslInitControlLibrary** initializes the Control library with the **XFaceMaker** graphic interface editor or interpreted **XFaceMaker** applications. This initialization function returns nonzero if it succeeds and zero if it fails.

This function is obsolete since the Control widgets are always included in XFaceMaker. It has been included in the distribution for backward compatibility.

## EXAMPLE

## SCOPE

Application.

## SEE ALSO

## NAME

**XnslInitDrawLibrary**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslFm [libraries]

int XnslInitDrawLibrary (w)
Widget   w;
```

## DESCRIPTION

The function **XnslInitDrawLibrary** initializes the Draw library with the **XFaceMaker** graphic interface editor or interpreted **XFaceMaker** applications. This initialization function returns nonzero if it succeeds and zero if it fails.

## EXAMPLE

## SCOPE

Application.

## SEE ALSO

## NAME

**XnslInitHtml**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslHtmlFm [libraries]

void XnslInitTree()
```

## DESCRIPTION

The function **XnslInitHtml** initializes the XnslHtmlFm library with the **XFaceMaker** graphic interface editor or interpreted **XFaceMaker** applications.

## EXAMPLE

## SCOPE

Application.

## SEE ALSO

## NAME

**XnslInitTree**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslTreeFm [libraries]

void XnslInitTree()
```

## DESCRIPTION

The function **XnslInitTree** initializes the XnslTree library with the **XFaceMaker** graphic interface editor or interpreted **XFaceMaker** applications.

## EXAMPLE

## SCOPE

Application.

## SEE ALSO

## NAME

**XnslInitGraphLibrary**

## SYNOPSIS

```
cc [flags] files [libraries] -lXrt2dFm [libraries]

void XnslInitGraphLibrary ()
```

## DESCRIPTION

The function **XnslInitGraphLibrary** initializes the Graph library with the **XFaceMaker** graphic interface editor or interpreted **XFaceMaker** applications.

## EXAMPLE

## SCOPE

Application.

## SEE ALSO

## NAME

**XnslInitXrtField**

## SYNOPSIS

```
cc [flags] files [libraries] -lXrtFieldFm [libraries]

void XnslInitXrtField ()
```

## DESCRIPTION

The function XnslInitXrtField initializes the XrtFieldFm library with the **XFaceMaker** graphic interface editor or interpreted **XFaceMaker** applications.

## EXAMPLE

## SCOPE

Application.

## SEE ALSO

## NAME

**XnslInitXrtGear**

## SYNOPSIS

```
cc [flags] files [libraries] -lXrtGearFm [libraries]

void XnslInitXrtField ()
```

## DESCRIPTION

The function XnslInitXrtGearinitializes the XrtGearFm library with the **XFaceMaker** graphic interface editor or interpreted **XFaceMaker** applications.

## EXAMPLE

## SCOPE

Application.

## SEE ALSO

## NAME

**XnslInitXrtTable**

## SYNOPSIS

```
cc [flags] files [libraries] -lXrtTableFm [libraries]

void XnslInitXrtTable ()
```

## DESCRIPTION

The function XnslInitXrtTable initializes the XrtTableFm library with the **XFaceMaker** graphic interface editor or interpreted **XFaceMaker** applications.

## EXAMPLE

## SCOPE

Application.

## SEE ALSO

## 3.1 FACE Convenience Functions

### 3.1.1 General Purpose Functions

The **XnslStand** library provides convenience functions for easier access to functions and structures in the NSL Extension libraries. While these functions are mainly intended for use with **FACE** scripts, the application may also call them directly from C.

The functions listed in this section are in the XnslStand library or in the XnslCtrl library, as indicated in the function's reference page.

## NAME

**FmEventTime**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslStand [libraries]

#include <Xnsl/Stand.h>

Time FmEventTime (event)
XEvent              *event;
```

## DESCRIPTION

**FmEventTime** returns the time field from the given *event* structure. If the type of the event does not have a time field, **FmEventTime** returns **CurrentTime**.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**FmFontListGetCharSet**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslStand [libraries]

#include <Xnsl/Stand.h>

XmStringCharSet FmFontListGetCharSet (list)
XmFontList         list;
```

## DESCRIPTION

**FmFontListGetCharSet** returns the character set for the first font in the fontlist *list*.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**FmFontListGetFont**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslStand [libraries]

#include <Xnsl/Stand.h>

Font FmFontListGetFont (list)
XmFontList list;
```

## DESCRIPTION

**FmFontListGetFont** returns the X11 font ID of the first font in the fontlist *list*.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**FmFontListGetFontStruct**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslStand [libraries]

#include <Xnsl/Stand.h>

XFontStruct * FmFontListGetFontStruct (list)
XmFontList          list;
```

## DESCRIPTION

**FmFontListGetFontStruct** returns an X11 font structure containing the description of the first font in the fontlist *list*.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**FmFontStructToFontList**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslStand [libraries]

#include <Xnsl/Stand.h>

XmFontList FmFontStructToFontList (fs)
XFontStruct        *fs;
```

## DESCRIPTION

**FmFontStructToFontList** takes the X11 font structure *fs* and returns a font list with that font as its only entry. The character set used is the simple character set as returned by **FmGetSimpleCharSet**.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**FmFontToFontList**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslStand [libraries]

#include <Xnsl/Stand.h>

XmFontList FmFontToFontList (w, fid)
Widget    w;
Font      fid;
```

## DESCRIPTION

**FmFontToFontList** builds an **XmFontList** from the given X11 font ID *fid*.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**FmGetSimpleCharSet**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslStand [libraries]

#include <Xnsl/Stand.h>

XmStringCharSet FmGetSimpleCharSet ();
```

## DESCRIPTION

**FmGetSimpleCharSet** returns the character set used by the OSF/Motif function **XmStringCreateSimple**.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**FmGrabPointer**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslStand [libraries]

#include <Xnsl/Stand.h>

int FmGrabPointer (w, owner, moving, cursor, event)
Widget              w;
Boolean             owner;
Boolean             moving;
String              cursor;
XEvent              *event;
```

## DESCRIPTION

**FmGrabPointer** calls the X Intrinsics routine **XtGrabPointer** with the widget *w*. It also passes the events's flag *owner* unchanged. It always passes GrabModeAsync for both the pointer and keyboard modes, and never asks to confine the cursor to the widget. The flag *moving* determines which events to select in the event mask. If *moving* is **True**, **FmGrabPointer** selects pointer motion (including motion hints) and button release events, otherwise it selects only button release events. It converts the string in *cursor* to an X cursor using the standard converter, and extracts the time from the X event pointed to by event. The value returned is that of XtGrabPointer.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XtGrabPointer

## NAME

**FmKillTimeOut**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslCtrl [libraries]

#include <Xnsl/Types.h>

void FmKillTimeOut (timer)
TimeOut            *timer;
```

## DESCRIPTION

**FmKillTimeOut** removes a timeout that was started by **FmStartTimeOut**. *timer* is the handle on the timeout that was returned by **FmStartTimeOut**.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

FmStartTimeOut

## NAME

**FmStartTimeOut**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslCtrl [libraries]

#include <Xnsl/Types.h>

TimeOut * FmStartTimeOut (w, callback, interval)
Widget                    w;
String                    callback;
unsigned int              interval;
```

## DESCRIPTION

**FmStartTimeOut** arranges to call the widget *w*'s callback list for the resource named *callback* at regular intervals. The parameter *callback* contains the name of the resource of the callback list, and *interval* contains the number of milliseconds between each call to the callback list. **FmStartTimeOut** returns a pointer to an opaque structure that the application passes to **FmKillTimeOut** to remove the timeout.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

FmKillTimeOut

## NAME

**FmUngrabPointer**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslStand [libraries]

#include <Xnsl/Stand.h>

void FmUngrabPointer (w, event)
Widget              w;
XEvent              *event;
```

## DESCRIPTION

**FmUngrabPointer** releases the grabbed pointer by calling **XtUngrab-Pointer** with the widget *w* and the time extracted from the X event pointed to by *event*.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

### 3.1.2 The XRT/graph Widget

The **XnslGraph** library provides some convenience functions for easier access to the data structures and functions of the **XRT/graph** Widget. See the **XRT/graph** documentation for complete details of data structures, data types, and other concepts specific to the **XRT/graph** Widget.

## NAME

**XrtAppendPoint**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

Boolean XrtAppendPoint (data, set, x, y);
XrtData       *data;
int           set;
float         x;
float         y;
```

## DESCRIPTION

Append the specified point to *data*. If *data* is of type **XRT_DATA_GENERAL**, the new *x*-value is *x* and the new *y*-value is *y*. If *data* is of type **XRT_DATA_ARRAY**, the new *x*-value is *x* and *set* is ignored, and *y, ...* is a list of *y*-values for all the sets. Note however, that functions called in FACE may have no more than twelve arguments. Thus, if *data* has more than nine sets of data, **XrtAppendPoint** may not be called in FACE code, but may still be called in C code, provided that the correct number of values are present. For data with more than nine sets, FACE code can call **XrtArrDataAppendPts** with an array of *y*-values. It can create the array by converting a string to the type "floatTable" (**XnslFloatTable). XrtAppendPoint** returns **True** if it succeeds, else **False**.

Starting with XRT/Graph vers. 3.0.0, use the function **XrtDataHandleAppendPoint** instead.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtDataHandleAppendPoint

**NAME**

**XrtCreateMapResult**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

XrtMapResult * XrtCreateMapResult ();
```

**DESCRIPTION**

Return an **XrtMapResult** structure for use with the function **XrtMap** or with FACE convenience functions.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

XrtFreeMapResult

## NAME

**XrtCreatePickResult**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

XrtPickResult * XrtCreatePickResult ();
```

## DESCRIPTION

Return an **XrtPickResult** structure for use with the function **XrtPick** or with FACE convenience functions.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtFreePickResult

## NAME

**XrtCreateTextDesc**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

XrtTextDesc * XrtCreateTextDesc ();
```

## DESCRIPTION

Return an **XrtTextDesc** structure for use with **XRT/graph** text description functions or with FACE convenience functions.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtFreeTextDesc

## NAME

**XrtDataHandleAppendPoint**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

Boolean XrtDataHandleAppendPoint (data, set, x, y);
XrtDataHandle data;
int           set;
float         x;
float         y;
```

## DESCRIPTION

Append the specified point to *data*. If *data* is of type **XRT_DATA_GENERAL**, the new *x*-value is *x* and the new *y*-value is *y*. If *data* is of type **XRT_DATA_ARRAY**, the new *x*-value is *x* and *set* is ignored, and *y, ...* is a list of *y*-values for all the sets. Note however, that functions called in FACE may have no more than twelve arguments. Thus, if *data* has more than nine sets of data, **XrtDataHandleAppendPoint** may not be called in FACE code, but may still be called in C code, provided that the correct number of values are present. For data with more than nine sets, FACE code can call **XrtArrDataAppendPts** with an array of *y*-values. It can create the array by converting a string to the type "floatTable" (**XnslFloatTable). XrtDataHandleAppendPoint** returns **True** if it succeeds, else **False**.

This function should be used instead of XrtAppendPoint for XRT/graph vers. 3.0.0 and up.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtDataHandleGetNumberOfPoints**

## SYNOPSIS

```
int XrtDataHandleGetNumberOfPoints (data, set);
XrtDataHandle data;
int           set;
```

## DESCRIPTION

Return the number of points in a set within *data*. If *data* is of type **XRT_DATA_GENERAL**, then *set* is the set number. If *data* is of type **XRT_DATA_ARRAY**, then *set* is ignored (since all sets have the same number of points).

This function can be used to replace **XrtDataGetNumberOfPoints** for XRT/graph version 3.0.0 and up, in FACE scripts.

Your application code should use the equivalent KLGroup function, **XrtDataGetNPoints**. You can call **XrtDataGetNPoints** also in your FACE scripts, instead of **XrtDataHandleGetNumberOfPoints**.

## EXAMPLE

## SCOPE

FACE script

## SEE ALSO

XrtDataGetNPoints XrtDataGetNumberOfPoints

## NAME

**XrtDataHandleGetNumberOfSets**

## SYNOPSIS

```
int XrtDataHandle GetNumberOfSets (data);
XrtDataHandle data;
```

## DESCRIPTION

Return the number of sets (*i.e.*, lines) in *data*, regardless of its data type.

This function can be used to replace **XrtDataGetNumberOfSets** for XRT/graph version 3.0.0 and up, in FACE scripts.

Your application code should use the equivalent KLGroup function, **XrtDataGetNSets**. You can call **XrtDataGetNSets** also in your FACE scripts, instead of **XrtDataHandleGetNumberOfSets**.

## EXAMPLE

## SCOPE

FACE script

## SEE ALSO

XrtDataGetNSets XrtDataGetNumberOfSets

## NAME

**XrtDataHandleGetXValue**

## SYNOPSIS

```
float XrtDataHandleGetXValue (data, set, point);
XrtDataHandle *data;
int           set;
int           point;
```

## DESCRIPTION

Return the *x*-value of the point specified in *data*.If *data* is of type **XRT_DATA_GENERAL**, then *set* is the set number and *point* is the point number. If *data* is of type **XRT_DATA_ARRAY**, then *set* is ignored and *point* is the point number.

This function can be used to replace **XrtDataGetXValue** for XRT/graph version 3.0.0 and up, in FACE scripts.

Your application code should use the equivalent KLGroup function, **XrtDataGetXElement**. You can call **XrtDataGetXElement** also in your FACE scripts, instead of **XrtDataHandleGetXValue**.

## EXAMPLE

## SCOPE

FACE script

## SEE ALSO

XrtDataGetXElement XrtDataGetXValue

## NAME

**XrtDataHandleGetYValue**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

float XrtDataHandleGetYValue (data, set, point);
XrtDataHandle *data;
int           set;
int           point;
```

## DESCRIPTION

Return the *y*-value of the point specified in *data* whose set number is *set* and whose point number is *point*.

This function can be used to replace **XrtDataGetYValue** for XRT/graph version 3.0.0 and up, in FACE scripts.

Your application code should use the equivalent KLGroup function, **XrtDataGetYElement**. You can call **XrtDataGetYElement** also in your FACE scripts, instead of **XrtDataHandleGetYValue**.

## EXAMPLE

## SCOPE

FACE script

## SEE ALSO

XrtDataGetYElement XrtDataGetYValue

## NAME

**XrtDataHandleOutputData**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

Boolean XrtDataHandleOutputData (data, name, over-
write);
XrtDataHandle data;
String        name;
Boolean       overwrite;
```

## DESCRIPTION

Write the data in ***data*** to the file *name*.Return **True** upon success and **False** upon failure. If **XrtDataHandleOutputData** fails, the program may obtain a brief error message by calling **XrtGetError**. If the file exists*, overwrite* must be **True** or **XrtDataHandleOutputData** returns **False** and writes nothing.

This function can be used to replace **XrtOutputData** for XRT/graph version 3.0.0 and up.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtOutputData

## NAME

**XrtDataHandleSetPoint**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtDataHandleSetPoint (data, set, point, x, y);
XrtDataHandle data;
int           set;
int           point;
float         x;
float         y;
```

## DESCRIPTION

Set the *x*- and *y*-values of the point specified in *data* whose set number is *set* and whose point number is *point*. The new *x*-value is *x* and the new *y*-value is *y*.If *data* is of type **XRT_DATA_ARRAY** and *set* is –1, then *y, ...* is a list of *y*-values for all the sets. Note however, that functions called in FACE may have no more than twelve arguments. Thus, if *data* has more than eight sets of data, *set* may not be –1 in FACE code, but may still be –1 in C code, provided that the correct number of values are present. **XrtDataHandleSetPoint** returns a non-zero value if it succeeds, else 0.

This function can be used to replace **XrtDataSetPoint** for XRT/graph version 3.0.0 and up.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtDataSetPoint

## NAME

**XrtDataHandleSetXValue**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

Boolean XrtDataHandleSetXValue (data, set, point,
value);
XrtDataHandle data;
int           set;
int           point;
float         value;
```

## DESCRIPTION

Set the *x*-value of the point specified in *data* to the given *value*. If *data* is of type **XRT_DATA_GENERAL**, then *set* is the set number and *point* is the point number. If *data* is of type **XRT_DATA_ARRAY**, then *set* is ignored and *point* is the point number. **XrtDataHandleSetXValue** returns non-zero if it succeeds, else 0.

This function can be used to replace **XrtSetXValue** for XRT/graph version 3.0.0 and up.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtSetXValue

## NAME

**XrtDataHandleSetYValue**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtDataHandleSetYValue (data, set, point,
value);
XrtDataHandle data;
int           set;
int           point;
float         value;
```

## DESCRIPTION

Set the *y-value* of the point specified in *data* whose set number is *set* and whose point number is *point*. The new value is *value*. If *data* is of type **XRT_DATA_ARRAY** and *set* is –1, then *value, ...* is a list of values for all the sets. Note however, that functions called in FACE may have no more than twelve arguments. Thus, if *data* has more than nine sets of data, *set* may not be –1 in FACE code, but may still be –1 in C code, provided that the correct number of values are present. **XrtDataHandleSetYValue** returns non-zero if it succeeds, else 0.

This function can be used to replace **XrtSetYValue** for XRT/graph version 3.0.0 and up.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtSetYValue

## NAME

**XrtFmDrawPS**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

Boolean XrtFmDrawPS (w, name, overwrite);
Widget        w;
String        name;
int           overwrite;
```

## DESCRIPTION

Call **XrtDrawPS** for the widget *w*, saving the PostScript output in the file *name*. Return **True** upon success and **False** upon failure. If **XrtFmDrawPS** fails, the program may obtain a brief error message by calling **XrtGetError**. If the file exists, *overwrite* must be non-zero or **XrtFmDrawPS** returns **False** and writes nothing.
Make sure that the XRTPostScript file parameters have been initialized before calling this function. To initialize the parameters, you can call **XrtResetDefaultPS.**

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

The **XRT/graph** documentation for complete details of data structures, data types, and other concepts specific to the **XRT/graph** Widget.
The section “Functions Common to XRT/graph and XRT/3d” on page 198 for functions that set parameters for *XrtFmDrawPS*, e.g. XrtResetDefaultPS

## NAME

**XrtFreeMapResult**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

void XrtFreeMapResult (map);
XrtMapResult  *map;
```

## DESCRIPTION

Free the **XrtMapResult** structure *map* previously allocated by **XrtCreateMapResult**.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtCreateMapResult

## NAME

**XrtFreePickResult**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

void XrtFreePickResult (pick);
XrtPickResult *pick;
```

## DESCRIPTION

Free the **XrtPickResult** structure *pick* previously allocated by **XrtCreatePickResult**.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtCreatePickResult

## NAME

**XrtFreeTextDesc**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

void XrtFreeTextDesc (text);
XrtTextDesc   *text;
```

## DESCRIPTION

Free the **XrtTextDesc** structure *text* previously allocated by **XrtCreateTextDesc**.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtCreateTextDesc

## NAME

**XrtGetNumberOfPoints**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtGetNumberOfPoints (data, set);
XrtData       *data;
int           set;
```

## DESCRIPTION

Return the number of points in a set within *data*. If *data* is of type **XRT_DATA_GENERAL**, then *set* is the set number. If *data* is of type **XRT_DATA_ARRAY**, then *set* is ignored (since all sets have the same number of points). This function is obsolete with XRT/graph version 3.0.0 and up.

Starting with XRT/graph vers. 3.0.0, you should use the function **XrtDataHandleGetNumberOfPoints** or **XrtDataGetNPoints** instead.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtDataHandleGetNumberOfPoints XrtDataGetNPoints

## NAME

**XrtGetNumberOfSets**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtGetNumberOfSets (data);
XrtData       *data;
```

## DESCRIPTION

Return the number of sets (*i.e.*, lines) in *data*, regardless of its data type. This function is obsolete with XRT/graph version 3.0.0 and up.

Starting with XRT/graph vers. 3.0.0, you should use the function **XrtDataHandleGetNumberofSets** or **XrtDataGetNSets** instead.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtDataHandleGetNumberOfSets XrtDataGetNSets

## NAME

**XrtGetTextAdjust**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

XrtAdjust XrtGetTextAdjust (text);
XrtTextDesc   *text;
```

## DESCRIPTION

Return the type of adjustment for the text description structure pointed to by *text*.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtSetTextAdjust

## NAME

**XrtGetTextAnchor**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

XrtAnchor XrtGetTextAnchor (text);
XrtTextDesc   *text;
```

## DESCRIPTION

Return the type of anchoring for the text description structure pointed to by *text*.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtSetTextAnchor

## NAME

**XrtGetTextBackground**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

String XrtGetTextBackground (text);
XrtTextDesc   *text;
```

## DESCRIPTION

Return as a **String** the name of the background color for the text description structure pointed to by *text*.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtGetTextForeground

## NAME

**XrtGetTextBorderType**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

XrtBorder XrtGetTextBorderType (text);
XrtTextDesc   *text;
```

## DESCRIPTION

Return the border type for the text description structure pointed to by *text*.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtGetTextBorderWidth**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtGetTextBorderWidth (text);
XrtTextDesc   *text;
```

## DESCRIPTION

Return the border width for the text description structure pointed to by *text*.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtGetTextConnected**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

Boolean XrtGetTextConnected (text);
XrtTextDesc   *text;
```

## DESCRIPTION

Return the current value of the *connected* flag for the text description structure pointed to by *text*.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtGetTextCoordHeight**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtGetTextCoordHeight (text);
XrtTextDesc   *text;
```

## DESCRIPTION

Return the height in the *coords* structure for the text description structure pointed to by *text*.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtGetTextCoordWidth

## NAME

**XrtGetTextCoordWidth**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtGetTextCoordWidth (text);
XrtTextDesc   *text;
```

## DESCRIPTION

Return the width in the *coords* structure for the text description structure pointed to by *text*.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtGetTextCoordHeight

## NAME

**XrtGetTextCoordX**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtGetTextCoordX (text);
XrtTextDesc   *text;
```

## DESCRIPTION

Return the *x* pixel position of the *coords* structure for the text description structure pointed to by *text*.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtGetTextCoordY

## NAME

**XrtGetTextCoordY**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtGetTextCoordY (text);
XrtTextDesc   *text;
```

## DESCRIPTION

Return the *y* pixel position of the *coords* structure for the text description structure pointed to by *text*.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtGetTextCoordX

## NAME

**XrtGetTextDataSet**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

XrtDsGroup XrtGetTextDataSet (text);
XrtTextDesc   *text;
```

## DESCRIPTION

Return the data set for the text position of the text description structure pointed to by *text*. Its attachment type must be **XRT_TEXT_ATTACH_VALUE**, **XRT_TEXT_ATTACH_DATA**, or **XRT_TEXT_ATTACH_DATA_VALUE**.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtGetTextFont**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

Font XrtGetTextFont (text);
XrtTextDesc   *text;
```

## DESCRIPTION

Return the X11 font ID for the text description structure pointed to by *text*. This function is obsolete for XRT/graph vers 3.0.0 and up. It is replaced by **XrtGetTextFontList**.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtGetTextFontList

## NAME

**XrtGetTextFontList**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

XmFontList XrtGetTextFontList (text);
XrtTextDesc   *text;
```

## DESCRIPTION

Return the XmFontList for the text description structure pointed to by *text*.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtGetTextFont

## NAME

**XrtGetTextForeground**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

String XrtGetTextForeground (text);
XrtTextDesc   *text;
```

## DESCRIPTION

Return as a **String** the name of the foreground color for the text description structure pointed to by *text*.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtGetTextBackground

## NAME

**XrtGetTextOffset**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtGetTextOffset (text);
XrtTextDesc   *text;
```

## DESCRIPTION

Return the offset for the text description structure pointed to by *text*.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtSetTextOffset

## NAME

**XrtGetTextPixelX**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtGetTextPixelX (text);
XrtTextDesc   *text;
```

## DESCRIPTION

Return the *x* pixel value for the text position of the text description structure pointed to by *text*. Its attachment type must be **XRT_TEXT_ATTACH_PIXEL**.
Otherwise, returns -1.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtGetTextPixelY

## NAME

**XrtGetTextPixelY**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtGetTextPixelY (text);
XrtTextDesc   *text;
```

## DESCRIPTION

Return the *y* pixel value for the text position of the text description structure pointed to by *text*. Its attachment type must be **XRT_TEXT_ATTACH_PIXEL**.
Otherwise, returns -1.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtGetTextPixelX

## NAME

**XrtGetTextPoint**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtGetTextPoint (text);
XrtTextDesc   *text;
```

## DESCRIPTION

Return the point number for the text position of the text description structure pointed to by *text*. Its attachment type must be **XRT_TEXT_ATTACH_DATA** or **XRT_TEXT_ATTACH_DATA_VALUE**.
Otherwise, returns -1.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtGetTextPositionType**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

XrtAttachType XrtGetTextPositionType (text);
XrtTextDesc   *text;
```

## DESCRIPTION

Return the type of attachment for the text description structure pointed to by *text*.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtGetXValue**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

float XrtGetXValue (data, set, point);
XrtData       *data;
int           set;
int           point;
```

## DESCRIPTION

Return the *x*-value of the point specified in *data*.If *data* is of type **XRT_DATA_GENERAL**, then *set* is the set number and *point* is the point number. If *data* is of type **XRT_DATA_ARRAY**, then *set* is ignored and *point* is the point number. This function is obsolete with XRT/graph version 3.0.0 and up.

Starting with XRT/graph vers. 3.0.0, you should use the function **XrtDataHandleGetXValue** or **XrtDataGetXElement** instead.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtDataHandleGetXValue XrtDataGetXElement

## NAME

**XrtGetYValue**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

float XrtGetYValue (data, set, point);
XrtData       *data;
int           set;
int           point;
```

## DESCRIPTION

Return the *y*-value of the point specified in *data* whose set number is *set* and whose point number is *point*. This function is obsolete with XRT/graph version 3.0.0 and up.

Starting with XRT/graph vers. 3.0.0, you should use the function **XrtDataHandleGetYValue** or **XrtDataGetYElement** instead.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtDataHandleGetYValue XrtDataGetYElement

## NAME

**XrtMapEvent**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

XrtRegion XrtMapEvent (w, group, event, map);
Widget        w;
XrtDsGroup    group;
XEvent        *event;
XrtMapResult  *map;
```

## DESCRIPTION

Call **XrtMap** with the *x* and *y* pixel values in the X event pointed to by *event*. The arguments *w*and *group* are passed unchanged. If *map* is **XrtMapResult (0)**, it is replaced by a pointer to a static structure. The program may access this static structure in the other FACE convenience functions that require an **XrtMapResult** structure in the same way by passing **XrtMapResult (0). (**Note that this will ***not*** work with **XrtMap** itself*.)* If map is not **XrtMapResult (0)**, it is passed unchanged to **XrtMap**. In *"Eval"* FACE scripts in translation tables, the pointer to the event is referenced by a dollar sign ($).

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

**NAME**

**XrtMapXPixel**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtMapXPixel (map);
XrtMapResult  *map;
```

**DESCRIPTION**

Return the *x* pixel value from the **XrtMapResult** structure *map*.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

## NAME

**XrtMapXValue**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtMapXValue (map);
XrtMapResult     *map;
```

## DESCRIPTION

Return the *x*-value from the **XrtMapResult** structure *map*.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtMapYAxis**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

XrtDsGroup XrtMapYAxis (map);
XrtMapResult  *map;
```

## DESCRIPTION

Return the data set number of the corresponding *y*-axis from the **XrtMapResult** structure *map*.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALS

## NAME

**XrtMapYPixel**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtMapYPixel (map);
XrtMapResult  *map;
```

## DESCRIPTION

Return the *y* pixel value from the **XrtMapResult** structure *map*.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

**NAME**

**XrtMapYValue**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtMapYValue (map);
XrtMapResult  *map;
```

**DESCRIPTION**

Return the *y*-value from the **XrtMapResult** structure *map*.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

## NAME

**XrtOutputData**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

Boolean XrtOutputData (data, name, overwrite);
XrtData       *data;
String        name;
Boolean       overwrite;
```

## DESCRIPTION

Write the data in the **XrtData** structure ***data*** to the file *name*.Return **True** upon success and **False** upon failure. If **XrtOutputData** fails, the program may obtain a brief error message by calling **XrtGetError**. The data written may be read by **XrtMakeDataFromFile**. If the file exists*, overwrite* must be **True** or **XrtOutputData** returns **False** and writes nothing. This function is obsolete with XRT/graph version 3.0.0 and up.

Starting with XRT/graph vers. 3.0.0, you should use the function **XrtDataHandleOutputData** instead.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtDataHandleOutputData

**NAME**

**XrtPickDataSet**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

XrtDsGroup XrtPickDataSet (pick);
XrtPickResult *pick;
```

**DESCRIPTION**

Return the data set (data group) from the **XrtPickResult** structure *pick*.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

## NAME

**XrtPickDistance**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtPickDistance (pick);
XrtPickResult *pick;
```

## DESCRIPTION

Return the distance from the **XrtPickResult** structure *pick*.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

**NAME**

**XrtPickEvent**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

XrtRegion XrtPickEvent (w, group, event, pick,
focus);
Widget        w;
XrtDsGroup    group;
XEvent        *event;
XrtPickResult *pick;
XrtFocus      focus;
```

**DESCRIPTION**

Call **XrtPick** with the *x* and *y* pixel values in the X event pointed to by *event*. The arguments *w*, *group*, and *focus* are passed unchanged. If *pick* is **XrtPickResult** (0), it is replaced by a pointer to a static structure. The program may access this static structure in the other FACE convenience functions that require an **XrtPickResult** structure in the same way by passing **XrtPickResult(0). (**Note that this will ***not*** work with **XrtPick** itself.*)* If *pick* is not **XrtPickResult (0)**, it is passed unchanged to **XrtPick**. In "Eval" FACE scripts in translation tables, the pointer to the event is referenced by a dollar sign ($).

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

## NAME

**XrtPickPoint**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtPickPoint (pick);
XrtPickResult *pick;
```

## DESCRIPTION

Return the point number from the **XrtPickResult** structure *pick*.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

**NAME**

**XrtPickSet**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtPickSet (pick);
XrtPickResult *pick;
```

**DESCRIPTION**

Return the set number from the **XrtPickResult** structure *pick*.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

## NAME

**XrtPickXPixel**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtPickXPixel (pick);
XrtPickResult *pick;
```

## DESCRIPTION

Return the *x* pixel value from the **XrtPickResult** structure *pick*.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

**NAME**

**XrtPickYPixel**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtPickYPixel (pick);
XrtPickResult *pick;
```

**DESCRIPTION**

Return the *y* pixel value from the **XrtPickResult** structure *pick*.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

## NAME

**XrtSetPoint**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

intXrtSetPoint (data, set, point, x, y);
XrtData       *data;
int           set;
int           point;
float         x;
float         y;
```

## DESCRIPTION

Set the *x*- and *y*-values of the point specified in *data* whose set number is *set* and whose point number is *point*. The new *x*-value is *x* and the new *y*-value is *y*.If *data* is of type **XRT_DATA_ARRAY** and *set* is –1, then *y, ...* is a list of *y*-values for all the sets. Note however, that functions called in FACE may have no more than twelve arguments. Thus, if *data* has more than eight sets of data, *set* may not be –1 in FACE code, but may still be –1 in C code, provided that the correct number of values are present. **XrtSetPoint** returns non-zero if it succeeds, else 0.

Starting with XRT/graph vers. 3.0.0, you should use the function **XrtDataHandleSetPoint** instead.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtDataHandleSetPoint

## NAME

**XrtSetTextAdjust**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

void XrtSetTextAdjust (text, adjust);
XrtTextDesc   *text;
XrtAdjust     adjust;
```

## DESCRIPTION

Set the type of adjustment for the text description structure pointed to by *text*.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtSetTextAnchor**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

void XrtSetTextAnchor (text, anchor);
XrtTextDesc   *text;
XrtAnchor     anchor;
```

## DESCRIPTION

Set the type of anchoring for the text description structure pointed to by *text*.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtSetTextBorder**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

void XrtSetTextBorder (text, border, width);
XrtTextDesc   *text;
XrtBorder     border;
int           width;
```

## DESCRIPTION

Set the border type and width for the text description structure pointed to by *text*.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtSetTextColors**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

void XrtSetTextColors (text, fg, bg);
XrtTextDesc   *text;
String        fg;
String        bg;
```

## DESCRIPTION

Set the foreground color to *fg* and the background color to *bg* for the text description structure pointed to by *text*. Note that these values are **String**s, not **Pixel**s.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtSetTextConnected**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

void XrtSetTextConnected (text, okay);
XrtTextDesc   *text;
int           okay;
```

## DESCRIPTION

Set the *connected* flag for the text description structure pointed to by *text*.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtSetTextData**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

void XrtSetTextData (text, group, set, point);
XrtTextDesc   *text;
XrtDsGroup    group;
int           set;
int           point;
```

## DESCRIPTION

Set the data set to *group*, set number to *set*, and point number to *point* for the position of the text description structure pointed to by *text*. Its attachment type must be **XRT_TEXT_ATTACH_DATA**.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtSetTextDataValue**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

void XrtSetTextDataValue (text, group, set, point,
y);
XrtTextDesc   *text;
XrtDsGroup    group;
int           set;
int           point;
double        y;
```

## DESCRIPTION

Set the data set to *group*, set number to *set*, point number to *point*, and *y*-value to *y* for the position of the text description structure pointed to by *text*. Its attachment type must be **XRT_TEXT_ATTACH_DATA_VALUE**.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtSetTextFont**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

void XrtSetTextFont (text, fid);
XrtTextDesc   *text;
Font          fid;
```

## DESCRIPTION

Set the X11 font to the font ID *fid* for the text description structure pointed to by *text*.

This function is obsolete with XRT/graph version 3.0.0 and up, because Font resources have been changed to XmFontList. Additionally, the XrtTextDesc structure has been replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtSetTextFontList

## NAME

**XrtSetTextFontList**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

void XrtSetTextFontList (text, fid);
XrtTextDesc   *text;
XmFontList    fid;
```

## DESCRIPTION

Set the XmFontList to *fid* for the text description structure pointed to by *text*.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtSetTextOffset**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

void XrtSetTextOffset (text, offset);
XrtTextDesc   *text;
int           offset;
```

## DESCRIPTION

Set the *offset* for the text description structure pointed to by *text*.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtSetTextPixel**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

void XrtSetTextPixel (text, x, y);
XrtTextDesc   *text;
int           x;
int           y;
```

## DESCRIPTION

Set the *x* and *y* pixel values to *x* and *y* for the position of the text description structure pointed to by *text*. Its attachment type must be **XRT_TEXT_ATTACH_PIXEL**.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtSetTextPosition**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

void XrtSetTextPosition (text, type, ....);
XrtTextDesc   *text;
XrtAttachType type;
```

## DESCRIPTION

Set the text position in the text description structure pointed to by *text*. The type of attachment is passed in `type`. The meaning of the additional arguments depend on the value of `type`

- **XRT_TEXT_ATTACH_PIXEL**
  The x and y pixel values (type **int**) to attach to.
- **XRT_TEXT_ATTACH_VALUE**
  The data set (type **XrtDsGroup)** and x- and y-values (type **Float**) to attach to.
- **XRT_TEXT_ATTACH_DATA**
  The data set (type **XrtDsGroup),** set number (type **int**), and point number (type **int**) to attach to.
- **XRT_TEXT_ATTACH_DATA_VALUE**
  The data set (type **XrtDsGroup),** set number (type **int**), and point number (type **int**), and y-value (type **Float**) to attach to.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtSetTextPsFont**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

void XrtSetTextPsFont (text, name, size);
XrtTextDesc   *text;
String        name;
int           size;
```

## DESCRIPTION

Set the PostScript font and size for the text description structure pointed to by *text*.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtSetTextStringList**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

void XrtSetTextStringList (text, list);
XrtTextDesc   *text;
String        *list;
```

## DESCRIPTION

Set the text strings for the text description structure pointed to by *text*. The argument *list* points to a list of strings terminated by a NULL pointer.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtSetTextStrings**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

void XrtSetTextStrings (text, ...);
XrtTextDesc   *text;
String        ...;
```

## DESCRIPTION

Set the text strings for the position of the text description structure pointed to by *text*. The strings are set to the values in the argument list, which must end with **String (0)**. Since FACE function calls may have up to twelve arguments, a FACE script may set at most ten strings with this function. If a script needs to set more than ten text strings, it should use **XrtSetTextStringList**. Of course, C code can set any number of strings.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtSetTextValue**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

void XrtSetTextValue (text, group, x, y);
XrtTextDesc   *text;
XrtDsGroup    group;
double        x;
double        y;
```

## DESCRIPTION

Set the data set to *group* and the *x*-and *y*-values for the position of the text description structure pointed to by *text*. Its attachment type must be **XRT_TEXT_ATTACH_VALUE**.

Starting with XRT/graph vers. 3.0.0 this function should not be used as the XrtTextDesc structure is replaced with a new API similar to that of XRT/3d.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtSetXValue**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtSetXValue (data, set, point, value);
XrtData       *data;
int           set;
int           point;
float         value;
```

## DESCRIPTION

Set the *x*-value of the point specified in *data* to the given *value*. If *data* is of type **XRT_DATA_GENERAL**, then *set* is the set number and *point* is the point number. If *data* is of type **XRT_DATA_ARRAY**, then *set* is ignored and *point* is the point number. **XrtSetXValue** returns non-zero if it succeeds, else 0.

Starting with XRT/graph vers. 3.0.0, you should use the function **XrtDataHandleSetXValue** instead.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtDataHandleSetXValue

## NAME

**XrtSetYValue**
(obsolete)

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslGraph [libraries]

#include <Xnsl/XrtGraphFm.h>

int XrtSetYValue (data, set, point, value);
XrtData       *data;
int           set;
int           point;
float         value;
```

## DESCRIPTION

Set the *y-value* of the point specified in *data* whose set number is *set* and whose point number is *point*. The new value is *value*. If *data* is of type **XRT_DATA_ARRAY** and *set* is –1, then *value, ...* is a list of values for all the sets. Note however, that functions called in FACE may have no more than twelve arguments. Thus, if *data* has more than nine sets of data, *set* may not be –1 in FACE code, but may still be –1 in C code, provided that the correct number of values are present. **XrtSetYValue** returns non-zero if it succeeds, else 0.

Starting with XRT/graph vers. 3.0.0, you should use the function **XrtDataHandleSetYValue** instead.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

XrtDataHandleSetYValue

### 3.1.3 The XRT/3d Widget

The **Xnsl3d** library provides these convenience functions for easier access to functions and structures provided by the **XRT/3d** Widget.

## NAME

**Xnsl3dComplete**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

void Xnsl3dComplete ()
```

## DESCRIPTION

The function **Xnsl3dComplete** has been defined for consistency with other extension libraries. It does nothing.

## EXAMPLE

## SCOPE

Application.

## SEE ALSO

XnslInit3dLibrary.

**NAME**

**Xrt3dContourGetFillColor**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

String Xrt3dContourGetFillColor (cs);
Xrt3dContourStyle*cs;
```

**DESCRIPTION**

Return the name of the fill color in the contour style pointed to by *cs*.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

## NAME

**Xrt3dContourGetLineColor**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

String Xrt3dContourGetLineColor (cs);
Xrt3dContourStyle*cs;
```

## DESCRIPTION

Return the name of the line color in the contour style pointed to by *cs*.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

**NAME**

**Xrt3dContourGetLinePattern**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

Xrt3dLinePattern Xrt3dContourGetLinePattern (cs);
Xrt3dContourStyle*cs;
```

**DESCRIPTION**

Return the line pattern in the contour style pointed to by *cs*.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

## NAME

**Xrt3dContourGetLineWidth**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

int Xrt3dContourGetLineWidth (cs);
Xrt3dContourStyle*cs;
```

## DESCRIPTION

Return the line width in the contour style pointed to by *cs*.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

**NAME**

**Xrt3dContourSetFillColor**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

void Xrt3dContourSetFillColor (cs, fill);
Xrt3dContourStyle*cs;
String        fill;
```

**DESCRIPTION**

Set the fill color in the contour style pointed to by *cs* to the color named in *fill*.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

## NAME

**Xrt3dContourSetLineColor**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

void Xrt3dContourSetLineColor (cs, line);
Xrt3dContourStyle*cs;
String        line;
```

## DESCRIPTION

Set the line color in the contour style pointed to by *cs* to the color named in *line*.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

**NAME**

**Xrt3dContourSetLinePattern**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

void Xrt3dContourSetLinePattern (cs, pattern);
Xrt3dContourStyle*cs;
Xrt3dLinePatternpattern;
```

**DESCRIPTION**

Set the line pattern in the contour style pointed to by *cs* to *pattern*.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

## NAME

**Xrt3dContourSetLineWidth**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

void Xrt3dContourSetLineWidth (cs, width);
Xrt3dContourStyle*cs;
int           width;
```

## DESCRIPTION

Set the line width in the contour style pointed to by *cs* to *width*.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

**NAME**

**Xrt3dCreateContourStyle**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

Xrt3dContourStyle * Xrt3dCreateContourStyle (fill,
line, width, pattern);
String        fill;
String        line;
int           width;
Xrt3dLinePatternpattern;
```

**DESCRIPTION**

Create an **Xrt3dContourStyle** data structure with the given fill color, line color, line width, and line pattern, and return a pointer to the new structure.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

Xrt3dFreeContourStyle

## NAME

**Xrt3dCreateMapResult**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

Xrt3dMapResult * Xrt3dCreateMapResult ();
```

## DESCRIPTION

Allocate a new **Xrt3dMapResult** structure and return a pointer to it. The structure is uninitialized.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

Xrt3dFreeMapResult

**NAME**

**Xrt3dCreatePickResult**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

Xrt3dPickResult * Xrt3dCreatePickResult ();
```

**DESCRIPTION**

Allocate a new **Xrt3dPickResult** structure and return a pointer to it. The structure is uninitialized.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

Xrt3dFreePickResult

## NAME

**Xrt3dFmDrawPS**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

Boolean Xrt3dFmDrawPS (w, name, overwrite);
Widget        w;
String        name;
Boolean       overwrite;
```

## DESCRIPTION

Call **Xrt3dDrawPS** for the given Widget and the named file.If the file exists, do not overwrite it unless *overwrite* is **True**. If the attempt succeeded, return **True**, otherwise **False**. The application can call **XrtGetError** to obtain a brief error message explaining why an attempt failed. If the file exists,and *overwrite* is **False, Xrt3dFmDrawPS** returns **False** and writes nothing.

Make sure that the XRTPostScript file parameters have been initialized before calling this function. To initialize the parameters, you can call **XrtResetDefaultPS.**

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

The **XRT/3d** documentation for complete details of data structures, data types, and other concepts specific to the **XRT/3d** Widget.

The section “Functions Common to XRT/graph and XRT/3d” on page 198 for functions that set parameters for *Xrt3dFmDrawPS*, e.g. XrtResetDefaultPS

**NAME**

**Xrt3dFreeContourStyle**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

void Xrt3dFreeContourStyle (cs);
Xrt3dContourStyle*cs;
```

**DESCRIPTION**

Free an **Xrt3dContourStyle** data structure created by **Xrt3dCreateContourStyle**.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

Xrt3dCreateContourStyle

**NAME**

**Xrt3dFreeMapResult**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

void Xrt3dFreeMapResult (map);
Xrt3dMapResult*map;
```

**DESCRIPTION**

Free an **Xrt3dMapResult** structure allocated by **Xrt3dCreateMapResult**.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

Xrt3dCreateMapResult

**NAME**

**Xrt3dFreePickResult**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

void Xrt3dFreePickResult (pick);
Xrt3dPickResult*pick;
```

**DESCRIPTION**

Free an **Xrt3dPickResult** structure allocated by **Xrt3dCreatePickResult**.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

Xrt3dCreatePickResult

## NAME

**Xrt3dGetDataType**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

Xrt3dDataType Xrt3dGetDataType (data);
Xrt3dData     *data;
```

## DESCRIPTION

Return the type of *data*. At this time, the only type supported by the **XRT/3d** widget is **XRT3D_DATA_GRID.**

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

**NAME**

**Xrt3dGetNoValue**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

float Xrt3dGetNoValue (data);
Xrt3dData     *data;
```

**DESCRIPTION**

Return the value used to indicate no value in *data*.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

## NAME

**Xrt3dGetNumXLines**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

int Xrt3dGetNumXLines (data);
Xrt3dData     *data;
```

## DESCRIPTION

Return the number of *x* lines (contained in the field *numx*) in *data*.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

**NAME**

**Xrt3dGetNumYLines**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

int Xrt3dGetNumYLines (data);
Xrt3dData     *data;
```

**DESCRIPTION**

Return the number of *y* lines (contained in the field *numy*) in *data*.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

## NAME

**Xrt3dGetValue**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

float Xrt3dGetValue (data, x, y);
Xrt3dData     *data;
int           x;
int           y;
```

## DESCRIPTION

Return the value of the point indexed by (*x*, *y*).Both indices are relative to zero.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

**NAME**

**Xrt3dGetXOrigin**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

float Xrt3dGetXOrigin (data);
Xrt3dData     *data;
```

**DESCRIPTION**

Return the *x*-value of the origin (contained in the field *xorig*) in *data*.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

**NAME**

**Xrt3dGetXStep**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

float Xrt3dGetXStep (data);
Xrt3dData     *data;
```

**DESCRIPTION**

Return the increment between *x* lines (contained in the field *xstep*) in *data*.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

**NAME**

**Xrt3dGetYOrigin**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

float Xrt3dGetYOrigin (data);
Xrt3dData     *data;
```

**DESCRIPTION**

Return the *y*-value of the origin (contained in the field *yorig*) in *data*.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

**NAME**

**Xrt3dGetYStep**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

float Xrt3dGetYStep (data);
Xrt3dData     *data;
```

**DESCRIPTION**

Return the increment between *y* lines (contained in the field *ystep*) in *data*.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

## NAME

**Xrt3dMapEvent**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

Xrt3dRegion Xrt3dMapEvent (w, event, map);
Widget         w;
XEvent         *event;
Xrt3dMapResult*map;
```

## DESCRIPTION

Call **Xrt3dMap** with the *x*- and *y*-values in the given X *event*. Store the result in *map*, or if *map* is zero, in a static structure provided by the FACE convenience routines. The following structure access routines also access this static structure if the given pointer is zero.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**Xrt3dMapXPixel**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

int Xrt3dMapXPixel (map);
Xrt3dMapResult*map;
```

## DESCRIPTION

Return the *x*-pixel value for *map*. If `map` is Xrt3dMapResult(0), the static structure is accessed.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

Xrt3dMapEvent

**NAME**

**Xrt3dMapXValue**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

float Xrt3dMapXValue (map);
Xrt3dMapResult*map;
```

**DESCRIPTION**

Return the graph's *x*-value for *map*. If `map` is Xrt3dMapResult(0), the static structure is accessed.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

Xrt3dMapEvent

## NAME

**Xrt3dMapYPixel**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

int Xrt3dMapYPixel (map);
Xrt3dMapResult*map;
```

## DESCRIPTION

Return the *y*-pixel value for *map*. If *map* is Xrt3dMapResult(0), the static structure is accessed.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

Xrt3dMapEvent

**NAME**

**Xrt3dMapYValue**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

float Xrt3dMapYValue (map);
Xrt3dMapResult*map;
```

**DESCRIPTION**

Return the graph's *y*-value for *map*. If `map` is Xrt3dMapResult(0), the static structure is accessed.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

Xrt3dMapEvent

## NAME

**Xrt3dMapZValue**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

float Xrt3dMapZValue (map);
Xrt3dMapResult*map;
```

## DESCRIPTION

Return the graph's *z*-value for *map*. If `map` is Xrt3dMapResult(0), the static structure is accessed.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

Xrt3dMapEvent

**NAME**

**Xrt3dOutputData**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

Boolean Xrt3dOutputData (data, name, overwrite);
Xrt3dData     *data;
String        name;
Boolean       overwrite;
```

**DESCRIPTION**

Output the given *data* to the file named *name*. If the file exists, *overwrite* must be **True** to overwrite the file.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

## NAME

**Xrt3dPickDistance**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

int Xrt3dPickDistance (pick);
Xrt3dPickResult*pick;
```

## DESCRIPTION

Return the distance from the indexed point for *pick*. If `pick` is Xrt3dPickResult(0), the static structure is accessed.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

Xrt3dPickEvent

## NAME

**Xrt3dPickEvent**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

Xrt3dRegion Xrt3dPickEvent (w, event, pick);
Widget        w;
XEvent        *event;
Xrt3dPickResult*pick;
```

## DESCRIPTION

Call **Xrt3dPick** with the *x*- and *y*-values in the given X *event*. Store the result in *pick*, or if *pick* is zero, in a static structure provided by the FACE convenience routines. The following structure access routines also access this static structure if the given pointer is zero.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**Xrt3dPickXIndex**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

int Xrt3dPickXIndex (pick);
Xrt3dPickResult*pick;
```

## DESCRIPTION

Return the *x*-index for *pick*. If `pick` is Xrt3dPickResult(0), the static structure is accessed.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

Xrt3dPickEvent

## NAME

**Xrt3dPickXPixel**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

int Xrt3dPickXPixel (pick);
Xrt3dPickResult*pick;
```

## DESCRIPTION

Return the *x*-pixel value for *pick*. If `pick` is Xrt3dPickResult(0), the static structure is accessed.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

Xrt3dPickEvent

## NAME

**Xrt3dPickYIndex**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

int Xrt3dPickYIndex (pick);
Xrt3dPickResult*pick;
```

## DESCRIPTION

Return the *y*-index for *pick*. If `pick` is Xrt3dPickResult(0), the static structure is accessed.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

Xrt3dPickEvent

## NAME

**Xrt3dPickYPixel**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

int Xrt3dPickYPixel (pick);
Xrt3dPickResult*pick;
```

## DESCRIPTION

Return the *y*-pixel value for *pick*. If `pick` is Xrt3dPickResult(0), the static structure is accessed.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

Xrt3dPickEvent

## NAME

**Xrt3dSetValue**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnsl3d [libraries]

#include <Xnsl/Xrt3dFm.h>

void Xrt3dSetValue (data, x, y, value);
Xrt3dData     *data;
int           x;
int           y;
float         value;
```

## DESCRIPTION

Set the point indexed by (*x*, *y*) to the given *value*.Both indices are relative to zero.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

### 3.1.4 Functions Common to XRT/graph and XRT/3d

The **XnslXrtPS** library provides some convenience functions common to both the **XRT/graph and XRT/3d widgets.**

**NAME**

**XrtGetError**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnslXrtPS [libraries]

#include <Xnsl/XrtXnslPs.h>

String XrtGetError ();
```

**DESCRIPTION**

Return a brief error message for a failed call to **XrtOutputData**, **XrtFmDrawPS**, or **Xrt3dFmDrawPS**.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

XrtOutputData XrtFmDrawPS Xrt3dFmDrawPS

#### 3.1.4.1 PostScript Output

The **XRT/graph and XRT/3d** widgets supply functions for writing a PostScript file of the graph represented by the widget. These functions have too many arguments for **FACE** scripts, so the convenience functions **XrtFmDrawPS** and **Xrt3dFmDrawPS** provide an alternate interface to the functions **XrtDrawPS** and **Xrt3dDrawPS**. The application calls the convenience functions listed here to set output parameters for the PostScript file. The **XRT/3d** widget does not use all of the parameters used by the **XRT/graph** widget, so you should consult the widget's documentation to see which parameters apply to your widget. The values set by these remain in effect for all subsequent calls to either **XrtFmDrawPS** or **Xrt3dFmDrawPS** until changed again by calling the corresponding convenience function.

## NAME

**XrtFillBackgroundPS**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslXrtPS [libraries]

#include <Xnsl/XrtXnslPs.h>

void XrtFillBackgroundPS (okay);
Boolean       okay;
```

## DESCRIPTION

Set the background flag parameter for **XRT**PostScript files.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

**NAME**

**XrtIsLandscapePS**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnslXrtPS [libraries]

#include <Xnsl/XrtXnslPs.h>

void XrtIsLandscapePS (okay);
Boolean       okay;
```

**DESCRIPTION**

Set the image orientation parameter for **XRT**PostScript files.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

## NAME

**XrtIsSmartMonoPS**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslXrtPS [libraries]

#include <Xnsl/XrtXnslPs.h>

void XrtIsSmartMonoPS (okay);
Boolean       okay;
```

## DESCRIPTION

Set the smart monochrome parameter for **XRT**PostScript files.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtResetDefaultPS**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslXrtPS [libraries]

#include <Xnsl/XrtXnslPs.h>

void XrtResetDefaultPS ();
```

## DESCRIPTION

Reset the **XRT**PostScript file parameters to their default values:

- Use inches: True
- Paper width: 8.5 (inches)
- Paper height: 11.0 (inches)
- Margin: 0.25 (inches)
- Landscape mode: False
- Image offset: (0.0, 0.0)
- Image size: $0.0 \times 0.0$
- Header font: Times-Bold
- Header size: 0
- Footer font: Times-Bold
- Footer size: 0
- Annotation font: Helvetica
- Annotation size: 0
- Legend font: Helvetica-Bold
- Legend size: 0
- Fill background: False
- Smart monochrome: False
- Show page: True

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

The **XRT/graph** or **XRT/3d** documentation for a complete explanation of these parameters.

## NAME

**XrtSetAnnoFontPS**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslXrtPS [libraries]

#include <Xnsl/XrtXnslPs.h>

void XrtSetAnnoFontPS (name, size);
String        name;
int           size;
```

## DESCRIPTION

Set the annotation font parameters for **XRT**PostScript files to the font named *name* with the given size.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtSetFooterFontPS**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslXrtPS [libraries]

#include <Xnsl/XrtXnslPs.h>

void XrtSetFooterFontPS (name, size);
String        name;
int           size;
```

## DESCRIPTION

Set the footer font parameters for **XRT**PostScript files to the font named *name* with the given size.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtSetHeaderFontPS**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslXrtPS [libraries]

#include <Xnsl/XrtXnslPs.h>

void XrtSetHeaderFontPS (name, size);
String        name;
int           size;
```

## DESCRIPTION

Set the header font parameters for **XRT**PostScript files to the font named *name* with the given size.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtSetImageOffsetPS**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslXrtPS [libraries]

#include <Xnsl/XrtXnslPs.h>

void XrtSetImageOffsetPS (x, y);
float         x;
float         y;
```

## DESCRIPTION

Set the image placement parameters for **XRT**PostScript files to the given (*x*, *y*) position.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtSetImageSizePS**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslXrtPS [libraries]

#include <Xnsl/XrtXnslPs.h>

void XrtSetImageSizePS (width, height);
Float         width;
float         height;
```

## DESCRIPTION

Set the image size parameters for **XRT**PostScript files to the given width and height.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtSetLegendFontPS**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslXrtPS [libraries]

#include <Xnsl/XrtXnslPs.h>

void XrtSetLegendFontPS (name, size);
String        name;
int           size;
```

## DESCRIPTION

Set the legend font parameters for **XRT**PostScript files to the font named *name* with the given size.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

## NAME

**XrtSetPaperPS**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslXrtPS [libraries]

#include <Xnsl/XrtXnslPs.h>

void XrtSetPaperPS (width, height, margin);
float         width;
float         height;
float         margin;
```

## DESCRIPTION

Set the paper size parameters for **XRT**PostScript files, using the given paper width, height, and margin.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

**NAME**

**XrtShowPagePS**

**SYNOPSIS**

```
cc [flags] files [libraries] -lXnslXrtPS [libraries]

#include <Xnsl/XrtXnslPs.h>

void XrtShowPagePS (okay);
Boolean       okay;
```

**DESCRIPTION**

Set the page display parameter for **XRT**PostScript files.

**EXAMPLE**

**SCOPE**

FACE script, application.

**SEE ALSO**

## NAME

**XrtUseInchesPS**

## SYNOPSIS

```
cc [flags] files [libraries] -lXnslXrtPS [libraries]

#include <Xnsl/XrtXnslPs.h>

void XrtUseInchesPS (okay);
Boolean       okay;
```

## DESCRIPTION

Set the unit parameter for **XRT**PostScript files. If *okay* is True, the values for paper size, margin, image offset, and image size are in inches. If False, the values are in centimeters. Changing the value of this parameter does not do any unit conversion, so it effectively changes the actual value for all these parameters.

## EXAMPLE

## SCOPE

FACE script, application.

## SEE ALSO

# CHAPTER 4: Building An Application

XFaceMaker lets you build three kinds of applications:

- a compiled application,
- an interpreted application,
- a customized interface editor.

The Extension libraries support all three kinds of applications.

## 4.1 The Xnsl Libraries

The NSL Extension libraries contain:

- libraries for the NSL widgets.
- a support library that provides a FACE interpreter for the **XnslDraw** widget.
- a library that provides common support routines for all widgets.
- a library that provides support for interpreting XFaceMaker files, including FACE scripts, and building a customized interface editor.
- libraries to support the XRT widgets.

| Library | Link Flag | Widgets or Description |
|---|---|---|
| **XnslFm** | –lXnslFm | **XFaceMaker** support. |
| **XnslCtrl** | –lXnslCtrl | **XnslBarGraph, XnslColorBox, XnslColorSB, XnslFontSelector, XnslIndicator**, **XnslJoystick**, **XnslMeter**, **XnslSlider**, **XnslStepper** |
| **XnslHtml** | –lXnslHtml | **VistaPoint** widget |
| **XnslTree** | –lXnslTree | **XnslTree** widget |
| **XnslDraw** | –lXnslDraw | **XnslDraw** and the Draw gadgets. |
| **XnslDf** | –lXnslDf | **FACE** interpreter for the **XnslDraw** widget. |
| **XnslTiff** | –lXnslTiff | Tiff library for the **XnslDraw** widget. |
| **XnslStand** | –lXnslStand | Common widget support code. |
| **XnslHtmlFm** | –lXnslTreeFm | **XFaceMaker** support for the **VistaPoint** widget. |
| **XnslTreeFm** | –lXnslTreeFm | **XFaceMaker** support for the **XnslTree** widget. |
| **XnslGraph** | –lXnslGraph | Convenience functions for the **XRT/graph** widget. |
| **Xnsl3d** | –lXnsl3d | Convenience functions for the **XRT/3d** widget. |
| **XnslXrtPS** | –lXnslXrtPs | Common widget support code for the **XRT/graph** and **XRT/3d** widgets. |

| Library | Link Flag | Widgets or Description |
|---|---|---|
| **Xrt2dFm** | –lXrt2dFm | **XFaceMaker** support for the **XRT/Graph** widget. |
| **Xrt3dFm** | –lXrt3dFm | **XFaceMaker** support for the **XRT/3d** widget. |
| **XrtTableFm** | –lXrtTableFm | **XFaceMaker** support for the **XRT/Table** and **XRT/Label** widgets. |
| **XrtGearFm** | –lXrtGearFm | **XFaceMaker** support for the **XRT/Gear** widgets. |
| **XrtFieldFm** | –lXrtFieldFm | **XFaceMaker** support for the **XRT/Field** widgets. |

The **XRT/graph**, **XRT/3d**, **XRT/Table**, **XRT/Label, XRT/Gear** and **XRT/Field** widgets do not come with the **Xnsl** libraries, but rather in their own libraries (invoked by **-lxrtm**, **-lxrt3d**, **-lxrttable, -lxrtgear,** and **-lxrtfield** respectively).

All widgets in the **Xnsl** libraries use the support library **XnslStand.** The **XnslDraw** widget also uses the **XnslDf** library and the math library (invoked by **–lm**). You need to link with the **XnslDf** library in compiled mode. In interpreted mode, the FACE interpreter is included in the **Fm** or **Fm_e** libraries. Only applications that load XFaceMaker (**.fm**) files and customized versions of the XFaceMaker interface editor require the XFaceMaker support library (XnslFm).

Each widget also comes with its corresponding **#include** file:

| Widget | File |
|---|---|
| **XnslBarGraph** | **<Xnsl/BarGraph.h>** |
| **XnslColorBox** | **<Xnsl/ColorBox.h>** |
| **XnslColorSB** | **<Xnsl/ColorSB.h>** |
| **XnslDraw** | **<Xnsl/DAll.h>** |
| **XnslFontSelector** | **<Xnsl/FontSelector.h>** |

| Widget | File |
|---|---|
| **XnslIndicator** | **<Xnsl/Indicator.h>** |
| **XnslJoystick** | **<Xnsl/Joystick.h>** |
| **XnslMeter** | **<Xnsl/Meter.h>** |
| **XnslSlider** | **<Xnsl/Slider.h>** |
| **XnslStepper** | **<Xnsl/Stepper.h>** |
| **XnslHtml** | **<Xnsl/Html.h>** |
| **XnslTree** | **<Xnsl/Tree.h>** |

Code that uses one of the common support routines without explicitly referencing a widget can include the file **<Xnsl/Stand.h>**. Code that uses one of the XFaceMaker support routines should include the file **<Xnsl/LibFm.h>**. For XRT widgets, for XFaceMaker support, you must include:

| Widget | File |
|---|---|
| **XRT/Graph** | **<Xnsl/XrtGraphFm.h>**<br>**<Xnsl/XrtXnslPs.h>** |
| **XRT/3d** | **<Xnsl/Xrt3dFm.h>**<br>**<Xnsl/XrtXnslPs.h>** |
| **XRT/Table** | **<Xnsl/XrtTableFm.h>** |

## 4.2 Building a Compiled Application

Your extended XFaceMaker executable will build your application skeletons for you (the xyzMain.c, xyzApp.c and xyzApp.h files). It will also create the Makefile to compile and link your executable: the default name of the compiled executable that is built is cxyz, for an interface file called xyz.fm or a project file called xyx.fmp. The command you type to create the compiled executable is

```
make -f xyzMake compiled
```

With this command, the Makefile will build the C code from the .fm file(s) and compile it; it will compile the xyzMain.c and xyzApp.c files. Refer to the XFaceMaker manuals for details on the application file skeletons generated by XFaceMaker. The Makefile generated by an extended XFaceMaker executable links your application with all the extension libraries with which the XFaceMaker executable was built.

Before you build your application, you should modify the xyzMain.c and xyzApp.c files as needed, just like for a standard XFaceMaker application: add the code in the xyzApp.c file for any application functions you call, attach non global active values, etc. Alternatively, you can modify the Makefile to link in any of your application specific libraries.

Additionally, if you are using the development (license-protected) versions of the NSL widget libraries, you should call the appropriate function to release the licenses used by your application before exiting your program: **XnslDrawComplete**, for the XDraw libray, **XnslHtmlComplete** for the VistaPoint widget library, and **XnslTreeComplete** for the XnslTree library. These functions release the license token for the development library. They take no arguments and have no return value. Application programs may call these routines when using the run-time (unprotected) version of the library , in which case they will do nothing.

The xyzMain.c file generated by XFaceMaker has the function calls to initialize the converters and to initialize the widgets as required. For example, in an xyzMain.c file built with **xfmwf** (this includes the XDraw widget extension and the NSL control widget extension), you would have:

```
XnslRegisterConverters();
XnslDrawInit(toplevel);
```

**XnslRegisterConverters** registers the converters for new resource types used in the NSL widgets in the Control library.

**XnslDrawInit** registers the converters for new resource types used in the XDraw library and it initializes the FACE interpreter that will interpret any animation scripts of drawings.

Similarly, if you have built your interface and application files using the **xfmwf** executable , your executable will be linked with the following libraries (plus any system libraries that are required on your platform):

```
-lFm_c -lXnslDraw -lXnslTiff -lXnslDf   -lXnslCtrl
-lXnslStand -lXm -lXt -lX11 -lm
```

If, now, you are working with an extended XFaceMaker executable that contains the following extensions:

XnslDraw XnslStand XnslHtml XnslTree Xrt3d XrtCommon XrtGear XrtField XrtGraph XrtTable

your xyzMain.c file will contain the following initialization lines:

```
XnslRegisterConverters();
XnslDrawInit(toplevel);
XtInitializeWidgetClass(xtXrt3dWidgetClass);
XtInitializeWidgetClass(xmXrtFieldWidgetClass);
XtInitializeWidgetClass(xtXrtGraphWidgetClass);
XtInitializeWidgetClass(xtXrtTableWidgetClass);
```

and your executable will be linked with the following libraries:

```
-lFm_c -lXnslDraw -lXnslTiff -lXnslDf -lXnsl3d -lXnslGraph
-lXnslCtrl -lXnslStand -lXnslHtml -lXnslTree -lXnslXrtPS
-lxrt3d -lxrtgear -lxrtfield -lxrtm -lxrttable -lXpm
-lXm -lXext -lXt -lX11 -lm
```

The XRT libraries will be enabled with ***xrt_auth***: this is run if at least one of the XRT extensions was used to build the XFaceMaker executable.

## 4.3 Building an Interpreted Application

If your application loads XFaceMaker files at run time, then it interprets these files at run time. Applications that interpret XFaceMaker files typically call **FmInitialize** to initialize XFaceMaker, then call **FmLoad** to load their files, and eventually call **FmLoop** to run the application. If your application interprets XFaceMaker files and also uses widgets in the NSL widget extesions, it must initialize the library after calling **FmInitialize**.

As for compiled applications, your extended XFaceMaker executable will build your application skeletons for you (the xyzMain.c, xyzApp.c and xyzApp.h files). It will also create the Makefile to compile and link your executable: the default name of the executable that is built is ixyz, for an interface file called xyz.fm. The command you type to create the interpreted executable is

```
make -f xyzMake interpreted
```

With this command, the Makefile will compile the xyzMain.c and xyzApp.c files. Refer to the XFaceMaker manuals for details on the application file skeletons generated by XFaceMaker. The Makefile generated by an extended XFaceMaker executable links your application with all the extension libraries with which the XFaceMaker executable was built.

Before you build your application, you should modify the xyzMain.c and xyzApp.c files as needed, just like for a standard XFaceMaker application: add the code in the xyzApp.c file for any application functions you call, attach non global active values, etc. Alternatively, you can modify the Makefile to link in any of your application specific libraries.

Additionally, if you are using the development (license-protected) versions of the NSL widget libraries, you should call the appropriate function to release the licenses used by your application before exiting your program: **XnslDrawComplete**, for the XDraw libray, **XnslHtmlComplete** for the VistaPoint widget library, and **XnslTreeComplete** for the XnslTree library. These functions release the license token for the development library. They take no arguments and have no return value. Application programs may call these routines when using the run-time (unprotected) version of the library , in which case they will do nothing.

The xyzMain.c contains the calls required to initialize each of the extension libraries. For example, if you have built your interface with the **xfmwf** executable, the xyzMain.c file will have:

```
XnslRegisterConverters();
XnslDrawInit(toplevel);
```

as in compiled applications, and also:

```
FmInitStand(FmGetFunctionsVector());
XnslInitControlLibrary();
XnslInitDrawLibrary(toplevel);
```

These functions are the widget initialization functions as they are declared in the extension definition files. Refer to the XFaceMaker documentation for further detail.

An interpreted application built with **xfmwf** will be linked with the following libraries (plus any system libraries that are required on your platform):

```
-lFm -lXnslFm -lXnslDraw -lXnslTiff -lXnslStand
-lXm -lXt -lX11 -lm
```

Continuing the example, if your application was built with an XFaceMaker executable that has the following extensions:

XnslDraw XnslStand XnslHtml XnslTree Xrt3d XrtCommon XrtGear XrtField XrtGraph XrtTable

your xyzMain.c file will contain the following initialization lines:

```
XnslRegisterConverters();
XnslDrawInit(toplevel);
XtInitializeWidgetClass(xtXrt3dWidgetClass);
XtInitializeWidgetClass(xmXrtToggleButtonWidgetClass);
XtInitializeWidgetClass(xmXrtFieldWidgetClass);
XtInitializeWidgetClass(xtXrtGraphWidgetClass);
XtInitializeWidgetClass(xtXrtTableWidgetClass);
```

as in compiled applications, and also:

```
FmInitStand(FmGetFunctionsVector());
XnslInitControlLibrary();
XnslInitDrawLibrary(toplevel);
XnslInitHtml();
XnslInitTree();
XnslInit3dLibrary();
XnslInitXrtField();
XnslInitGraphLibrary();
XnslInitXrtTable();
```

These functions are the widget initialization functions as they are declared in the extension definition files. Refer to the XFaceMaker documentation for further detail.

Your executable will be linked with the following libraries (plus any system libraries that are required on your platform):

```
-lFm -lXnslFm -lXnslDraw -lXnslTiff -lXnslHtmlFm
-lXnslTreeFm -lXrt3dFm -lXnsl3d -lXrtGearFm -lXrtFieldFm
-lXrt2dFm -lXnslGraph -lXrtTableFm -lXnslStand -lXnslHtml
-lXnslTree -lXnslXrtPS -lxrt3d -lxrtgear -lxrtfield -lxrtm
-lxrttable -lXpm -lXm -lXext -lXt -lX11 -lm
```

The XRT libraries will be enabled with ***xrt_auth***: this is run if at least one of the XRT extensions was used to build the XFaceMaker executable.

## 4.4 Building an Interface Editor

You can use any XFaceMaker executable to build a new extended XFaceMaker that contains the extensions you need. Before launching XFaceMaker, set any environment variables you will need (e.g. NSLHOME and XRTHOME if you plan to include XRT widget extensions). In the ***File*** menu, select the ***Load Extensions ...*** button and choose the extensions you want. Your new XFaceMaker executable is built automatically.

You can define new extensions, e.g. to add your own widgets. Refer to the *XFaceMaker User Guide* for instructions on adding new extensions.

# Index

# X